# BitGC Made (More) Efficient*

Wenhao Zhang† Hanlin Liu† Kang Yang‡ Wen-jie Lu§
Yu Yu¶‖ Xiao Wang† Chenkai Weng**

May 18, 2026

**Abstract**

Garbled circuits with one-bit-per-gate communication were recently introduced by Liu et al. (BitGC, Eurocrypt 2025), Meyer et al. (Crypto 2025), and Ishai et al. (Crypto 2025). However, these works focus primarily on the theoretical communication complexity, leaving open questions about practical computational efficiency. In this paper, we present a set of optimizations that substantially improve its practical efficiency. First, we eliminate key barriers to enable SIMD support for BitGC, leading to a substantial speedup in its homomorphic operations. Second, we demonstrate that XOR gates can be garbled without any communication, improving both efficiency and simplicity. Finally, we present a computationally efficient garbling scheme that requires zero communication for XOR gates and only 5 bits per AND gate. When applied to an AES-128 circuit, our fastest garbling scheme generates a garbled circuit of just 4 KB in 2 minutes on a single CPU core.

## 1 Introduction

Garbled circuit (GC) [Yao86] is a fundamental cryptographic tool that has been widely used to build secure two-party computation (2PC) [LP09], secure multi-party computation [BMR90], zero-knowledge proofs [JKO13], identity-based encryption [DG17], etc. It is particularly useful to build protocols with constant-round interaction and active security due to its inherent resilience against a corrupted evaluator.

Optimizing the size of GCs has been studied extensively both in practice and in a theoretical aspect. On the practical side, many optimizations have been proposed to improve the communication [BMR90, NPS99, KS08, PSSW09, KMR14, ZRE15, GLNP15, RR21] and computation [BHKR13, GKWY20, GKW+20] of garbling using only symmetric-key primitives. Notably, the scheme by Rosulek and Roy [RR21] achieves $1.5\lambda + 5$ bits per AND gate and no cost for XOR gates, where $\lambda$ is the security parameter. It was recently shown to be optimal [JRR25]. In contrast, asymptotically sublinear-size GCs aim for fully succinct constructions [GKP+13, GGH+13, BGG+14, Agr17], typically relying on heavy cryptographic primitives such as fully homomorphic encryption (FHE) or multilinear maps. However, these approaches remain completely impractical so far due to their non-blackbox use of FHE.

More recently, several works by Liu, Wang, Yang and Yu [LWYY25b], Meyer, Orlandi, Roy and Scholl [MORS25], and Ishai, Li and Lin [ILL25b] investigated rate-one garbling for Boolean circuits

*Full version of a paper published at Eurocrypt 2026 [ZLY+26].

†Northwestern University. **Email:** {wenhao.zhang, hanlin.liu}@northwestern.edu, wangxiao1254@gmail.com

‡State Key Laboratory of Cryptology. **Email:** yangk@sklc.org

§TikTok. **Email:** fionser@gmail.com

¶Shanghai Jiao Tong University. **Email:** yuyu@yuyu.hk

‖Shanghai Qi Zhi Institute.

**Arizona State University. **Email:** Chenkai.Weng@asu.edu

| | Communication | | | Computation | | |
|---|---|---|---|---|---|---|
| | AND | XOR | AES-128 | AND | XOR | AES-128 |
| BitGC-fast (Sec 5.1) | 5 bits | 0 bits | 4.0 KB | 16.24 ms | 0.43 ms | 116 s |
| BitGC-small (Sec 4) | 1 bit | 1 bit | 4.3 KB | 34.20 ms | 4.47 ms | 345 s |
| BitGC-tiny (Sec 5.2) | 1 bit | 0 bits | 0.8 KB | $\geq$ 331 ms | $\geq$ 44 ms | $\geq$ 56 min |
| BitGC [LWYY25b] | 1 bit | 1 bit | 4.3 KB | $\geq$ 1.45 s | $\geq$ 1.45 s | $\geq$ 14 hours |

Table 1: **Performance of our optimized** BitGC **variants compared to the original** BitGC **[LWYY25b].** An AES-128 circuit includes 6400 AND gates and 28176 XOR gates. The time costs of BitGC-fast and BitGC-small are benchmarked with single-threaded execution and those of BitGC-tiny and BitGC are lower bound estimations assuming the same experimental environment. The communication cost excludes data that is reusable across multiple garblings, and also omits the amortized communication of transmitting encrypted PRG seeds which is arbitrarily small.

with reasonable efficiency. In particular, these works have shown garbling schemes with one bit per gate communication based on lattice assumptions [LWYY25b, ILL25b] and classical assumptions [ILL25b, MORS25]. While these methods represent notable theoretical progress, their concrete efficiency is not clear and likely very far from being practical. Taking BitGC [LWYY25b] as an example, it requires offline homomorphic evaluation of a pseudorandom generator (PRG) as well as online computation of homomorphic gate assembling and complicated polynomial ring operations. Our conservative estimates show that a direct implementation of their scheme would require at least 14 hours to garble an AES-128 circuit.

### 1.1 Our Contribution

In this paper, we investigate the concrete efficiency of BitGC protocol and propose several important optimizations to maximize its performance. Our most practical scheme is over $400\times$ faster than the unoptimized BitGC with a reduced garbling size.

1. BitGC requires using somewhat homomorphic encryption (SWHE) to evaluate a PRG but cannot use the SIMD optimization to accelerate this step. This is because BitGC requires an even ciphertext space while SIMD requires primes friendly to Number Theoretic Transform (NTT). We observed that the above restriction is due to how BitGC correlates the garbled labels and the permutation bit; by revising the structure of BitGC, we are able to support any ciphertext space and hence bringing back SIMD to accelerate operations on SWHE.
2. Many circuits have been optimized to minimize the number of ANDs with possibly more XOR gates due to the free-XOR optimization [KS08]. However, all existing one-bit protocols treat all types of gates in the same way. We provide various kinds of free-XOR optimizations to BitGC reducing the communication (sometimes also computation) cost for handling XOR gates.
3. Although the offline homomorphic PRG evaluation can be SIMD-accelerated, the online garbling phase uses ring operations for each gate and cannot use such optimization. We propose a variant of BitGC with slightly more communication but avoiding the need for any homomorphic multiplication in the online phase.

By incorporating all these optimizations, we are able to obtain three schemes (BitGC-small, BitGC-tiny, and BitGC-fast) with different trade-offs as shown in Table 1. While all three schemes can enjoy the SIMD optimizations for improved offline phase, each of them has a different set of features.

- BitGC-small. This version is a direct optimization of the original scheme with SIMD acceleration and

slight optimization to make XOR gates cheaper in computation.

- BitGC-tiny. This version further eliminates the communication of XOR gates, but requires larger parameters for SWHE to support free-XOR. As a result, it has the smallest garbling but slow computation (still about $15\times$ faster than the unoptimized BitGC).
- BitGC-fast. This is the variant with cheapest computation and the most balanced. XOR gates are additionally $10\times$ cheaper to garble and requires no communication. The downside is that AND gates require $5$ bits of communication.

**Why not optimizing HSS-based constructions?** Ishai et al. [ILL25b] and Meyer et al. [MORS25] also proposed protocols for one bit garbling. Both protocols are built from homomorphic secret sharing (HSS) and semi-private HSS (a.k.a. algebraic homomorphic MAC [ILL25a]). By working directly over the HSS abstraction, they are able to instantiate one-bit garbling from a variety of assumptions. In this paper, we focus only on the practical efficiency of the garbling schemes and thus focus on lattice-based BitGC. Since these two works share some similar traits with BitGC, we believe some of our optimizations could be applied to their protocols.

# 2 Technical overview

Let $\mathcal{R}_p \stackrel{\text{def}}{=} \mathbb{Z}_p[X]/(X^n+1)$ and $\mathcal{R}_q \stackrel{\text{def}}{=} \mathbb{Z}_q[X]/(X^n+1)$. For any ring element $A$, we define $\mathsf{LSB}(A)$ as $A_0 \bmod 2$, where $A_0$ is the first coefficient of $A$.

## 2.1 Recap BitGC Construction

First, we give a brief summary of the BitGC construction [LWYY25b, LWYY25a]. In BitGC and many practical garbling schemes, the permutation bit on a wire is defined to be the LSB of the zero label on that wire. For clarity, we store the permutation bit explicitly; thus, each wire has a permutation bit and two garbled labels. The garbler picks a uniform global offset $\Delta \in \mathcal{R}_p$ with $\mathsf{LSB}(\Delta) = 1$. For any wire with $(A^0, A^1, \pi_a)$, the protocol requires $A^1 = A^0 + (-1)^{\pi_a} \cdot \Delta$. The BitGC sets $\pi_a = \mathsf{LSB}(A^0)$, which implies that $\mathsf{LSB}(A^{\pi_a}) = 0$ where $A^{\pi_a} = A^0 + (-1)^{\pi_a} \cdot \pi_a \cdot \Delta$. In what follows, $\Delta$ plays a dual role: it is both the global free-XOR offset on labels and the secret key under which the SWHE ciphertexts below are encrypted.

For a message $m$, we use $[\![m]\!]$ to denote an SWHE ciphertext of $m$ that is encrypted with a private key $\Delta$. Consider a gate $g$ with two inputs and one output. Given the input and output wire masks $\pi_a$, $\pi_b$, and $\pi_c$, the assembled garbled table consists of a set of three ciphertexts:

$$\begin{aligned} \tau_1 &= [\![(-1)^{\pi_c} \cdot (z_{1,0} - z_{0,0})]\!] \\ \tau_2 &= [\![(-1)^{\pi_c} \cdot (z_{0,1} - z_{0,0})]\!] \\ \tau_3 &= [\![(-1)^{\pi_c} \cdot (z_{0,0} + z_{1,1} - z_{1,0} - z_{0,1})]\!] \end{aligned} \tag{1}$$

where $z_{i,j} = g(i \oplus \pi_a, j \oplus \pi_b)$. Note that it implicitly defines $\tau_0 = 0$ due to the garbled row reduction [NPS99].

**Distributed evaluation of BitGC.** The BitGC scheme crucially requires a special homomorphism enabled by defining two functions Eval and Dec. We defer their instantiations to a later section and focus on their relations here. For any three ciphertexts $\tau_1, \tau_2, \tau_3$ and uniform $X, Y \in \mathcal{R}_p$, the following equation holds with overwhelming probability:

$$\begin{aligned} &\mathsf{Eval}_{i,j}(X + i \cdot \Delta, Y + j \cdot \Delta, \tau_1, \tau_2, \tau_3) - \mathsf{Eval}_{0,0}(X, Y, \tau_1, \tau_2, \tau_3) \\ &= i \cdot \mathsf{Dec}(\Delta, \tau_1) + j \cdot \mathsf{Dec}(\Delta, \tau_2) + i \cdot j \cdot \mathsf{Dec}(\Delta, \tau_3) \ . \end{aligned}$$

When assuming that $\mathsf{Dec}(\Delta, [\![m]\!]) = m \cdot \Delta$ and these $\tau$ values are ciphertexts as in Equation (1), the above equation can be further simplified to

$$\begin{aligned} &\mathsf{Eval}_{i,j}(X + i \cdot \Delta, Y + j \cdot \Delta, \tau_1, \tau_2, \tau_3) - \mathsf{Eval}_{0,0}(X, Y, \tau_1, \tau_2, \tau_3) \\ &= (-1)^{\pi_c} \cdot (z_{i,j} - z_{0,0}) \cdot \Delta \,. \end{aligned} \tag{2}$$

For any gate, the garbler always has all the input wire information $(A^0, A^1, \pi_a)$ and $(B^0, B^1, \pi_b)$, while the evaluator only holds the labels corresponding to the real values $v_a, v_b$ and masked values, namely $(A^{v_a}, \pi_a \oplus v_a)$ and $(B^{v_b}, \pi_b \oplus v_b)$. One can verify that the labels satisfy the correct correlation: for any bits $i$ and $j$, we have $A^i = A^{\pi_a} + (\pi_a \oplus i) \cdot \Delta$ and $B^j = B^{\pi_b} + (\pi_b \oplus j) \cdot \Delta$. Using Equation (2), it follows that

$$\begin{aligned} &\mathsf{Eval}_{\pi_a \oplus i, \pi_b \oplus j}(A^i, B^j, \tau_1, \tau_2, \tau_3) - \mathsf{Eval}_{0,0}(A^{\pi_a}, B^{\pi_b}, \tau_1, \tau_2, \tau_3) \\ &= (-1)^{\pi_c} \cdot (z_{\pi_a \oplus i, \pi_b \oplus j} - z_{0,0}) \cdot \Delta \,. \end{aligned}$$

However, the computation of $\pi_c$ encounters a deadlock: by definition, $\pi_c = \mathsf{LSB}(C^0)$ and one can compute labels $(C^0, C^1)$ from ciphertexts $(\tau_1, \tau_2, \tau_3)$, but these ciphertexts rely on $\pi_c$. To avoid the "chicken-and-egg" problem, the BitGC computes the permutation bit $\pi_c = \mathsf{LSB}(\mathsf{Eval}_{0,0}(A^{\pi_a}, B^{\pi_b}, \widetilde{\tau}_1, \widetilde{\tau}_2, \widetilde{\tau}_3)) \oplus g(\pi_a, \pi_b)$, where $\widetilde{\tau}_1, \widetilde{\tau}_2, \widetilde{\tau}_3$ are the same as $\tau_1, \tau_2, \tau_3$ except for replacing $(-1)^{\pi_c}$ with $(-1)^{r_c}$ for a uniform bit $r_c$. Then, the garbler sets $C^{z_{0,0}} = \mathsf{Eval}_{0,0}(A^{\pi_a}, B^{\pi_b}, \tau_1, \tau_2, \tau_3)$, and is able to compute $(C^0, C^1)$ with $(C^{z_{0,0}}, \Delta, \pi_c, z_{0,0})$. What the evaluator can compute is

$$\begin{aligned} &\mathsf{Eval}_{\pi_a \oplus v_a, \pi_b \oplus v_b}(A^{v_a}, B^{v_b}, \tau_1, \tau_2, \tau_3) \\ &= C^{z_{0,0}} + (-1)^{\pi_c} \cdot (z_{\pi_a \oplus v_a, \pi_b \oplus v_b} - z_{0,0}) \cdot \Delta \\ &= C^0 + (-1)^{\pi_c} \cdot z_{0,0} \cdot \Delta + (-1)^{\pi_c} \cdot (z_{\pi_a \oplus v_a, \pi_b \oplus v_b} - z_{0,0}) \cdot \Delta \\ &= C^0 + (-1)^{\pi_c} \cdot g(v_a, v_b) \cdot \Delta = C^{g(v_a, v_b)} = C^{v_c} \,, \end{aligned}$$

where $v_c = g(v_a, v_b)$ is the real bit on the output wire. The BitGC further shows that $\mathsf{LSB}(C^{v_c}) = \pi_c \oplus v_c$, which is the needed masked bit of the output wire. Now, the garbler has the output wire information $(C^0, C^1, \pi_c)$, and the evaluator holds $(C^{v_c}, \pi_c \oplus v_c)$. Thus, we can continue with the circuit evaluation.

**Transmitting BitGC garbled table.** First of all, transmitting encryptions of random bits is "free" in terms of communication, as the garbler can send an encrypted seed $[\![\boldsymbol{s}]\!]$ to the evaluator, and then both parties can locally and homomorphically expand it using a PRG to generate many pseudorandom bit encryptions. This does not need fully homomorphic encryption as long as a low-depth PRG such as [Gol11, Ale03, App12, BPR12, AL16, ABBS25] is used. Given that, one can transmit the garbled table for a gate with just one bit of communication.

Both garbler and evaluator start with encryptions of input permutation bits and a uniform bit, i.e., $[\![\pi_a]\!]$, $[\![\pi_b]\!]$ and $[\![r_c]\!]$. The garbler has input labels $(A^0, A^1)$, $(B^0, B^1)$. With this setup, the garbler garbles a gate in the following steps:

1. *Compute a candidate garbled table.* The garbler performs homomorphic evaluations on $[\![\pi_a]\!]$, $[\![\pi_b]\!]$ and $[\![r_c]\!]$ to obtain:
$$\begin{aligned} \widetilde{\tau}_1 &= [\![(-1)^{r_c} \cdot (z_{1,0} - z_{0,0})]\!] \\ \widetilde{\tau}_2 &= [\![(-1)^{r_c} \cdot (z_{0,1} - z_{0,0})]\!] \\ \widetilde{\tau}_3 &= [\![(-1)^{r_c} \cdot (z_{0,0} + z_{1,1} - z_{1,0} - z_{0,1})]\!] \end{aligned}$$
2. *Transmit the garbled-table bit.* The garbler computes
$$\pi_c = \mathsf{LSB}(\mathsf{Eval}_{0,0}(A^{\pi_a}, B^{\pi_b}, \widetilde{\tau}_1, \widetilde{\tau}_2, \widetilde{\tau}_3)) \oplus g(\pi_a, \pi_b)$$

and sends $d_c = r_c \oplus \pi_c$ to the evaluator. Both parties compute $[\![\pi_c]\!] = d_c \oplus [\![r_c]\!]$, and then can locally assemble the garbled table to obtain $\tau_i = (-1)^{d_c} \cdot \widetilde{\tau}_i$ for $i \in \{1, 2, 3\}$. In words, $\widetilde{\tau}_i$ is the candidate garbled-table ciphertext computed using the uniform bit $r_c$ in place of the still-unknown $\pi_c$, and the sign correction $(-1)^{d_c}$ converts each $\widetilde{\tau}_i$ into the actual garbled-table entry $\tau_i$ defined in Equation (1). As shown in the BitGC, setting the plaintext modulus $p$ to an even number ensures the correctness of this step.

3. *Obtain output labels.* The garbler computes $C^{z_{0,0}}$ as described above, from which the garbler can obtain both output labels, maintaining the invariant.

**Constructing distributed evaluation.** The BitGC presented two constructions of the Eval function under the Ring-LWE assumption [LPR10] and NTRU assumption [HPS98, PS21]. Both constructions use two basic operations: Dec and $\widehat{\mathsf{Dec}}$. Specifically, for any $X, Y \in \mathcal{R}_p$ and ciphertexts $\tau_1, \tau_2, \tau_3$, the Eval function is defined as:

$$\mathsf{Eval}_{i,j}(X, Y, \tau_1, \tau_2, \tau_3) \stackrel{\text{def}}{=} \mathsf{Dec}_i(X, \tau_1) + \mathsf{Dec}_j(Y, \tau_2) + \widehat{\mathsf{Dec}}_{i,j}(X, Y, \tau_3) \ .$$

For any ciphertext $\tau$ and uniform $X, Y \in \mathcal{R}_p$, the following properties hold with overwhelming probability:

1. Linear distributed decryption: $\mathsf{Dec}_1(X + \Delta, \tau) = \mathsf{Dec}_0(X, \tau) + \mathsf{Dec}(\Delta, \tau)$.
2. Correlated-key distributed decryption:

$$\widehat{\mathsf{Dec}}_{i,j}(X + i \cdot \Delta, Y + j \cdot \Delta, \tau) - \widehat{\mathsf{Dec}}_{0,0}(X, Y, \tau) = i \cdot j \cdot \mathsf{Dec}(\Delta, \tau) \ .$$

The linear distributed decryption function $\mathsf{Dec}_i$ above is based on [BKS19]. The BitGC scheme introduces the $\widehat{\mathsf{Dec}}_{i,j}$ function to securely multiply two additive shares. It defines an extended form of the GSW ciphertext and shows how to transform a standard ciphertext into this form. Finally, it applies linear distributed decryption to this extended ciphertext using key shares $(X \cdot Y, X, Y, 1)$. Independently, constructions of a similar spirit were also proposed by [ILL25b] and [MORS25]. They were proposed under slightly different formulations, but from a broader range of assumptions as well.

**Challenges in making BitGC practical.** Although the BitGC has a lot of potential, its current form is far from practical. Here, we summarize the greatest bottlenecks in concrete efficiency.

- *Incompatible with state-of-the-art SWHE optimizations.* The BitGC requires using an SWHE scheme to homomorphically evaluate a PRG. However, the scheme requires an even plaintext modulus $p$, which rules out SIMD techniques that rely on an NTT-friendly plaintext modulus. Enabling SIMD could potentially introduce three orders of magnitude improvement on homomorphic PRG evaluation.
- *Incompatible with free-XOR.* The majority of circuits are optimized to minimize the number of AND gates, due to the free-XOR technique [KS08], which makes XOR gates incur no communication cost and cheap in computational cost. On the other hand, the original BitGC scheme has the same cost across all types of gates; thus, using BitGC on existing free-XOR-optimized circuits would lead to suboptimal performance.
- *High ciphertext space and lattice dimension.* In addition, BitGC has some practical limitations that impact its performance. First, large parameter sizes are necessary due to the high homomorphic depth, specifically to support 1) homomorphic PRG evaluation and 2) homomorphic assembly of the garbled table. A larger modulus increases the cost of modular arithmetic, while a higher lattice dimension raises the cost of polynomial ring operations. Consequently, homomorphic evaluation remains inefficient, even with optimizations such as the Chinese Remainder Theorem.

Below, we will provide more details of our proposed optimizations and how they can be combined for more practical constructions.

## 2.2 Enabling SIMD Optimizations for Homomorphic PRG Evaluation

As mentioned above, one major bottleneck is the homomorphic evaluation of PRG over SWHE ciphertexts. Modern lattice-based SWHE schemes are highly efficient due to the use of SIMD optimization. In more detail, plaintext values are packed into polynomial ring elements, where the Number Theoretic Transform (NTT) is used, before being encrypted in SWHE. Due to the high lattice dimension, it is common to pack tens of thousands of plaintext messages into one ring element, thereby significantly amortizing the cost. This optimization requires an NTT-friendly prime as the plaintext modulus, but BitGC, as described above, requires an even plaintext space, thus making it incompatible with the packing technique. We first address this limitation.

**Why BitGC requires even plaintext modulus.** The root reason why BitGC requires an even plaintext modulus is the cyclic dependency among values:

1. Garbled-table ciphertexts $\tau$'s are dependent on the permutation bit $\pi_c$ by definition.
2. The output permutation bit $\pi_c$ is defined from the lowest bit of the output garbled label.
3. The output label is obtained by the garbled-table ciphertext (similar to the garbled row reduction technique [NPS99]).

In the BitGC, this is resolved by an alternative way to compute $\pi_c$, where the requirement on the plaintext space is introduced: the garbler instead computes a bit from the "candidate ciphertexts" (namely $\widetilde{\tau}_i$, where $\pi_c$ is replaced by a uniform bit $r_c$) as:

$$\mathsf{LSB}(\mathsf{Eval}_{0,0}(A^{\pi_a}, B^{\pi_b}, \widetilde{\tau}_1, \widetilde{\tau}_2, \widetilde{\tau}_3)) \oplus g(\pi_a, \pi_b) \ .$$

The correctness of this procedure can be argued as follows. When $r_c \neq \pi_c$, we have $\widetilde{\tau}_i = -1 \cdot \tau_i$, which means that $\mathsf{Eval}(A^{\pi_a}, B^{\pi_b}, \widetilde{\tau}_1, \widetilde{\tau}_2, \widetilde{\tau}_3) = -\mathsf{Eval}(A^{\pi_a}, B^{\pi_b}, \tau_1, \tau_2, \tau_3)$ when Eval is odd. Now, if the plaintext space is set to even, the resulting permutation bit is still the correct value because for any value $a$, as the lowest coefficient of the output of Eval, $(a \mod p) \mod 2 = (-a \mod p) \mod 2$ when $p$ is even.

**Avoiding even plaintext modulus.** BitGC breaks the cyclic dependency by computing the permutation bit without the $\tau$'s. Our observation, once writing the permutation bit separate from the garbled labels, is that we can also break the dependency by not enforcing the invariant that $\pi_c = \mathsf{LSB}(C^0)$. In our improved version, we pick all permutation bits uniformly; namely, $[\![\pi_a]\!]$, $[\![\pi_b]\!]$, and $[\![\pi_c]\!]$ are ciphertexts directly from the PRG. Now both parties can assemble the garbled-table ciphertexts $\tau_1, \tau_2, \tau_3$ directly.

Now, if we look at the values that both parties hold, they remain correlated in the correct manner. The garbler can compute $C^{z_{0,0}} = \mathsf{Eval}_{0,0}(A^{\pi_a}, B^{\pi_b}, \tau_1, \tau_2, \tau_3)$ and hence derive all values $(C^0, C^1, \pi_c)$ for the output wire. The evaluator can compute $C^{v_c} = \mathsf{Eval}_{\pi_a \oplus v_a, \pi_b \oplus v_b}(A^{v_a}, B^{v_b}, \tau_1, \tau_2, \tau_3)$. However, because this label does not embed the information of the masked bit, the garbler still needs to send one bit to the evaluator to transmit this information. Specifically, the garbler needs to send $\pi_c \oplus \mathsf{LSB}(C^0)$ to the evaluator, who computes the masked bit $\pi_c \oplus \mathsf{LSB}(C^0) \oplus \mathsf{LSB}(C^{v_c}) = v_c \oplus \pi_c$, thus maintaining the invariant.

## 2.3 Supporting Free-XOR

After the above steps of enabling SIMD, the permutation bits of all gates are uniformly random and independent of each other, and we hope further to eliminate the communication for XOR gates. Achieving free-XOR when labels are bit-strings is easy, since the relationship between zero- and one-labels is based on XOR: $A^1 = A^0 \oplus \Delta$. Therefore, in the traditional setting, given that $C^0 = A^0 \oplus B^0$, we have:

$$A^{v_a} \oplus B^{v_b} = (A^0 \oplus v_a\Delta) \oplus (B^0 \oplus v_b\Delta) = A^0 \oplus B^0 \oplus (v_a \oplus v_b)\Delta = C^0 \oplus v_c\Delta.$$

The masked bit is consistent for the same reason. However, in our context, our labels are defined as $A^1 = A^0 + (-1)^{\pi_a} \cdot \Delta$ and thus cannot be directly used in the above approach since it is not XOR homomorphic.

Below, we look at the BitGC scheme after changing it to support an odd modulus. For any gate function $g$, including the XOR function, two parties can obtain the correct correlation of labels. One bit is communicated so that the evaluator can obtain the masked bit, which is $v_c \oplus \pi_c = (v_a \oplus v_b) \oplus \pi_c$ in our context. We observe that this bit of communication can be eliminated if we can set $\pi_c = \pi_a \oplus \pi_b$, since the permutation bit itself is still "XOR-homomorphic" like a classical bit-string label. Therefore, the task of achieving no communication for XOR gates reduces to ensuring that the evaluator obtains correlated permutation bits: the output permutation bits of AND gates are uniform, while the output permutation bits of XOR gates are the XOR of input permutation bits.

Obtaining correlated permutation bits for free-XOR is not straightforward since the evaluator must obtain the SWHE ciphertexts of all bits. Here we discuss two possible ways, although we acknowledge that none of them are really practical:

- One can use a leveled HE scheme with a large ciphertext space to homomorphically compute the output permutation bits for all XOR gates. In this case, consecutive XOR operations can be evaluated in a binary tree manner. Since the noise does not carry across AND gates, an additional $O(\log N)$ levels of homomorphic multiplications are needed, where $N$ is the number of XOR gates in the longest sequence of consecutive XOR operations. This method allows zero communication for XOR gates, but in the worst case, the additional number of multiplication levels needed could be $O(\log |C|)$ where $|C|$ is the size of the circuit to be garbled.
- If an SWHE scheme with depth $L + d$ is provided, where $d$ is the depth needed by the original BitGC scheme, then one can achieve $O(1/L)$ bits per XOR gate for a layered circuit. In practice, this approach also leads to significant computational overhead, as increasing $L$ prohibitively increases the computational cost.

In conclusion, it is possible to achieve free communication for XOR gates, but the computational cost becomes even higher than that of AND gates. Nevertheless, in certain use cases where the garbling size is the most important, it could be of interest. Conceptually, it also enables us to obtain more realistic trade-offs, as described below.

### 2.4 Better Trade-Off between Computation and Communication

Our third major contribution is to obtain a more realistic trade-off. Note that after applying the SIMD optimization for evaluating the PRG, its cost would be significantly reduced. Now the bottleneck becomes 1) homomorphic assembling of the garbled-table ciphertexts (i.e., $\{\tau_i\}$), which requires two levels of homomorphic evaluation, and 2) distributed evaluation of the table by means of $\mathsf{Eval}_{i,j}$. Note that neither of these two parts can enjoy SIMD acceleration. The first step is dependent on the circuit structure and thus can only be packed if the same circuit needs to be garbled multiple times; the second step involves operations on polynomial rings and is not compatible with packing at all.

To reduce these costs, our goal is to eliminate the need for homomorphic multiplication completely after PRG evaluation. This way, we avoid the heavy operation of table assembling, and only need a smaller ciphertext modulus, which in turn reduces the dimension of the polynomial ring. We plan to accomplish it by slightly increasing the communication.

As an example, the first row of the garbled-table ciphertext $\tau_1$ is an encryption of $t_1 = (-1)^{\pi_c} \cdot (z_{1,0} - z_{0,0})$. One idea would be to first establish encryptions of some random $r$, where the evaluator only knows the ciphertext, and then the garbler sends $r + t_1$ to the evaluator. The evaluator can recover the encryption of $t_1$ with homomorphic addition. Note that $t_1$ can be any of $\{-1, 0, 1\}$, but the above addition works on a very

large plaintext domain. Thus $r$ can only be statistical masking with size at least $2^\rho$ for a statistical security parameter $\rho$. This would significantly increase the communication.

A better approach is to communicate the high and low bits of $t_1 + 1$, which range between 0 and 2, separately. Let $t_H, t_L$ be the high and low bits of $t_1 + 1$. Suppose the evaluator obtains the SWHE ciphertext of two uniform bits $r_H$ and $r_L$. The garbler can send $r_H \oplus t_H$ and $r_L \oplus t_L$ to the evaluator, who can compute $[\![t_H]\!] = [\![r_H]\!] \oplus (r_H \oplus t_H)$, and $[\![t_L]\!]$ in a similar way. Then the evaluator can assemble $[\![t_1]\!] = 2 \cdot [\![t_H]\!] + [\![t_L]\!] - 1$. Now, to communicate all three ciphertexts for each gate, the garbler sends 6 bits of communication and needs to use 6 bits from the homomorphically evaluated PRG. One more bit needs to be sent for the evaluator to recover the masked bit as described in Section 2.2, meaning that the communication cost is now 7 bits per gate. This is already a fairly competitive baseline, but below we show how to 1) further reduce the cost and 2) support free-XOR cheaply.

**A 5-bit version.** There is still hope for improvement because the above scheme is somewhat "wasteful". Each garbled-table ciphertext encrypts a value of three possibilities, while we transmit two bits that can encode four possibilities. Even without considering the correlation between the three values $(t_1, t_2, t_3)$, there are at most 27 possibilities, and thus, less than 6 bits of communication could still be possible. We look back at the plaintext in the garbled rows:

$$\begin{aligned} t_1 &= (-1)^{\pi_c} \cdot (z_{1,0} - z_{0,0}) \\ t_2 &= (-1)^{\pi_c} \cdot (z_{0,1} - z_{0,0}) \\ t_3 &= (-1)^{\pi_c} \cdot (z_{0,0} + z_{1,1} - z_{1,0} - z_{0,1}) \end{aligned} \tag{3}$$

Our goal is to come up with a minimum number of bits such that their additive linear combination (modulo a large value) produces $t_1, t_2, t_3$. One may consider decomposing them to small units as $(-1)^{\pi_c} \cdot z_{i,j}$'s but they still range in $\{-1, 0, 1\}$ and thus lead to a total of 8 bits. Instead, we crucially notice that

$$(-1)^{\pi_c} \cdot (z_{i,j} - z_{0,0}) = z_{i,j} \oplus \pi_c - z_{0,0} \oplus \pi_c \,.$$

Note that the right-hand side is a mix of arithmetic operations and Boolean operations. To see why this is true, we note that $(-1)^\pi \cdot (z - \pi) = z \oplus \pi$ for any bits $z, \pi$; thus we have:

$$\begin{aligned} (-1)^{\pi_c} \cdot (z_{i,j} - z_{0,0}) &= (-1)^{\pi_c} \cdot ((z_{i,j} - \pi_c) - (z_{0,0} - \pi_c)) \\ &= (-1)^{\pi_c} \cdot (z_{i,j} - \pi_c) - (-1)^{\pi_c} \cdot (z_{0,0} - \pi_c) \\ &= z_{i,j} \oplus \pi_c - z_{0,0} \oplus \pi_c \,. \end{aligned}$$

With this observation, one can transmit the garbled-table ciphertext with only 4 bits, sending the masked value of $z_{i,j} \oplus \pi_c$ with the evaluator holding the encrypted mask. The evaluator can homomorphically unmask and compute the difference with only linear operations. One more bit is still needed for the evaluator to obtain the masked bit for the output wire, totaling 5 bits for each gate.

In practice, this leads to significant improvement. Although using more PRG bits, it leads to a roughly $3\times$ improvement in overall computational cost (and about $30\times$ on the gate-dependent AND cost alone), due to elimination of homomorphic multiplications and reduction of lattice parameters.

**Lightweight free-XOR.** Finally, we would like to further improve this scheme so that the 5-bit communication is only needed for AND gates. We start by plugging the XOR gate function to $z_{i,j}$'s in Equation (3). Note that for a XOR gate, $z_{i,j} = g(i \oplus \pi_a, j \oplus \pi_b) = \pi_a \oplus \pi_b \oplus i \oplus j$. If we set $\pi_c = \pi_a \oplus \pi_b$ similar to Section 2.3, then $z_{i,j} \oplus \pi_c = i \oplus j$, and the plaintexts on three garbled rows can be simplified to

$$\begin{aligned} t_1 &= 1 \\ t_2 &= 1 \\ t_3 &= -2, \end{aligned}$$

which requires no communication. Essentially, for XOR gates, while Section 2.3 eliminates the communication for transmitting the masked value, here we eliminate the need to transmit the garbled tables. Furthermore, we no longer require the encryption of $\pi_c$ to assemble the garbled table and thus can avoid homomorphic multiplication over a high-depth circuit as needed in Section 2.3.

# 3 Preliminaries

## 3.1 Notation

We denote the computational and statistical security parameters by $\lambda$ and $\rho$, respectively. All logarithms are base 2, i.e., $\log(\cdot)$. For $n \in \mathbb{N}$, let $[n] \stackrel{\text{def}}{=} \{0, \ldots, n-1\}$. We write $x \xleftarrow{\$} \mathcal{S}$ for sampling $x$ uniformly at random from a set $\mathcal{S}$, and use $x \leftarrow \mathcal{D}$ to denote that sampling $x$ according to a distribution $\mathcal{D}$. Define the polynomial ring $\mathcal{R} \stackrel{\text{def}}{=} \mathbb{Z}[X]/(X^n+1)$ and, for a modulus $q \in \mathbb{N}^+$, the quotient ring $\mathcal{R}_q \stackrel{\text{def}}{=} \mathcal{R}/q\mathcal{R} = \mathbb{Z}_q[X]/(X^n+1)$. For a vector or polynomial $\boldsymbol{a}$, we denote its $i$-th component by $a_i$, with $a_0$ being the constant term. For a polynomial $\boldsymbol{a} \in \mathcal{R}$, let $\mathsf{LSB}(\boldsymbol{a}) = a_0 \bmod 2$. We use $\mathsf{negl}(\cdot)$ to represent a negligible function, where $\mathsf{negl}(\lambda) = o(\lambda^{-c})$ for every constant $c$. Additionally, $\mathsf{poly}(\cdot)$ denotes a polynomial function, with $\mathsf{poly}(\lambda) = O(\lambda^c)$ for some constant $c$.

For a Boolean circuit $f$, we use $\mathcal{I}$ to denote the set of circuit-input wires, and denote by $\mathcal{O}$ the set of circuit output wires. We use $\mathcal{W}$, $\mathcal{W}_{\mathsf{AND}}$ and $\mathcal{W}_{\mathsf{XOR}}$ to denote the sets of the output wires of all gates, AND gates and XOR gates, respectively. Each gate in a Boolean circuit is represented by a tuple $(\alpha, \beta, \gamma, T)$, where $\alpha$ and $\beta$ are the input wires, $\gamma$ is the output wire, and $T \in \{\mathsf{AND}, \mathsf{XOR}\}$ specifies the type of gate.

## 3.2 Definition of Garbling Schemes

We recall the definition of a garbling scheme from [RR21], which modifies the original definition [BHR12] in two ways: (i) correctness allows a negligible probability of failure, and (ii) authenticity is strengthened by giving the adversary an extra decoding information $d$. As in the previous work [BHR12], we define a side-information function $\Phi(\cdot)$ that deterministically maps a circuit $f$ to a string $\Phi(f)$ specifying which side information is revealed. In this work, we focus on the classical setting $\Phi_{\text{circ}}(f) = f$, where the entire circuit is revealed. The garbler executes the garbling algorithm $\mathsf{Gb}$ and encoding algorithm $\mathsf{En}$, while the evaluator runs the evaluation algorithm $\mathsf{Ev}$ and decoding algorithm $\mathsf{De}$. These algorithms are defined below.

**Definition 1** (Syntax)**.** *A garbling scheme consists of the following four polynomial-time algorithms:*

- $(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f)$*: takes as input a security parameter* $\lambda$ *and a circuit* $f$*, and outputs a garbled circuit* $F$*, an encoding information* $e$ *and a decoding information* $d$*.*
- $X \leftarrow \mathsf{En}(e, x)$*: takes as input* $e$ *and an input* $x$*, and outputs a garbled input* $X$*.*
- $Y \leftarrow \mathsf{Ev}(F, X)$*: takes as input* $F$ *and* $X$*, and outputs a garbled output* $Y$*.*
- $y \leftarrow \mathsf{De}(d, Y)$*: takes as input* $d$ *and* $Y$*, and outputs a circuit output* $y$*.*

**Correctness.** For any circuit $f$ and input $x$, for $(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f)$, then with probability $1 - \mathsf{negl}(\lambda)$,

$$\mathsf{De}\Big(d, \mathsf{Ev}\big(F, \mathsf{En}(e, x)\big)\Big) = f(x)\,.$$

**Privacy.** The privacy property ensures that the evaluator learns nothing about the input, even when given the transcript $(F, X, d)$. We formalize this as follows.

**Real World** $\mathsf{Exp}^{\mathsf{prv},\Phi}_{real}(1^\lambda, f, x)$

1. $(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f)$;
2. $X \leftarrow \mathsf{En}(e, x)$;
3. $b \leftarrow \mathcal{D}(F, X, d)$.

**Ideal World** $\mathsf{Exp}^{\mathsf{prv},\Phi}_{ideal,\mathcal{S}}(1^\lambda, f, x)$

1. $(F, X, d) \leftarrow \mathcal{S}(1^\lambda, \Phi(f), f(x))$
2. $b \leftarrow \mathcal{D}(F, X, d)$.

Figure 1: Experiments for privacy with respect to a side-information function $\Phi$.

**Real World** $\mathsf{Exp}^{\mathsf{obv},\Phi}_{real}(1^\lambda, f, x)$

1. $(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f)$;
2. $X \leftarrow \mathsf{En}(e, x)$;
3. $b \leftarrow \mathcal{D}(F, X)$.

**Ideal World** $\mathsf{Exp}^{\mathsf{obv},\Phi}_{ideal,\mathcal{S}}(1^\lambda, f, x)$

1. $(F, X) \leftarrow \mathcal{S}(1^\lambda, \Phi(f))$
2. $b \leftarrow \mathcal{D}(F, X)$.

Figure 2: Experiments for obliviousness w.r.t. a side-information function $\Phi$.

$\mathsf{Exp}^{\mathsf{auth}}_{\mathcal{A}}(1^\lambda, f, x)$

1. $(F, e, d) \leftarrow \mathsf{Gb}(1^\lambda, f)$; $X \leftarrow \mathsf{En}(e, x)$; $\widehat{Y} \leftarrow \mathcal{A}(F, X, d)$;
2. If $\mathsf{De}(d, \widehat{Y}) \neq \perp$ and $\widehat{Y} \neq \mathsf{Ev}(F, X)$, then output 1, else output 0.

Figure 3: Experiment for authenticity.

**Definition 2** (Privacy)**.** *A garbling scheme satisfies privacy if there exists a negligible function* $\mathsf{negl}(\cdot)$ *and a probabilistic polynomial-time (PPT) simulator* $\mathcal{S}$ *such that, for any probabilistic polynomial time (PPT) distinguisher* $\mathcal{D}$*, any function* $f$*, and any input* $x$*,*

$$\left|\Pr\left[\mathsf{Exp}^{\mathsf{prv},\Phi}_{real}(1^\lambda, f, x) = 1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{prv},\Phi}_{ideal,\mathcal{S}}(1^\lambda, f, x) = 1\right]\right| \leq \mathsf{negl}(\lambda)\,,$$

*where* $\mathsf{Exp}^{\mathsf{prv},\Phi}_{real}$ *and* $\mathsf{Exp}^{\mathsf{prv},\Phi}_{ideal,\mathcal{S}}$ *are defined in Figure 1.*

**Obliviousness.** Obliviousness ensures that the evaluator learns nothing about the inputs, even given $(F, X)$. Unlike privacy, the adversary does not receive the decoding information $d$ and thus cannot obtain the output $f(x)$. This property is useful when the evaluator should not learn the output, for example, when the garbling scheme is used as part of a larger system.

**Definition 3** (Obliviousness)**.** *A garbling scheme satisfies obliviousness if there exists a negligible function* $\mathsf{negl}(\cdot)$ *and a PPT simulator* $\mathcal{S}$ *such that, for any PPT distinguisher* $\mathcal{D}$*, any function* $f$*, and any input* $x$*,*

$$\left|\Pr\left[\mathsf{Exp}^{\mathsf{obv},\Phi}_{real}(1^\lambda, f, x) = 1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{obv},\Phi}_{ideal,\mathcal{S}}(1^\lambda, f, x) = 1\right]\right| \leq \mathsf{negl}(\lambda)\,,$$

*where* $\mathsf{Exp}^{\mathsf{obv},\Phi}_{real}$ *and* $\mathsf{Exp}^{\mathsf{obv},\Phi}_{ideal,\mathcal{S}}$ *are defined in Figure 2.*

**Authenticity.** Authenticity does not guarantee privacy: the adversary may learn the circuit $f$ and input $x$. However, it guarantees that the adversary cannot create a forged garbled output $\widehat{Y}$ that decodes successfully while differing from $\mathsf{Ev}(F, X)$, even when given $(F, X, d)$. This property is important for building constant-round zero-knowledge proofs (e.g., [JKO13, FNO15, KP17, HK20]).

**Definition 4** (Authenticity)**.** *For any PPT adversary $\mathcal{A}$, for any function $f$ and input $x$, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for any PPT adversary $\mathcal{A}$, the following holds:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{auth}}_{\mathcal{A}}(1^\lambda, f, x) = 1\right] \leq \mathsf{negl}(\lambda)\,,$$

*where* $\mathsf{Exp}^{\mathsf{auth}}_{\mathcal{A}}(1^\lambda, f, x)$ *is defined in Figure 3.*

### 3.3 SWHE with Distributed Decryption

We first recall the standard definition of somewhat homomorphic encryption (SWHE) schemes in the private-key setting. Then, we show the definition of a special property called distributed decryption (DD), and refer to SWHE with such a special property as SWHEwDD. Without loss of generality, we adopt two polynomial rings $\mathcal{R}$ and $\mathcal{R}_p$ in the following definition, and note that this definition can be directly extended to other finite fields or rings. We note that Liu et al. [LWYY25b] presented two instantiations of SWHEwDD under the RLWE and NTRU assumptions respectively, which satisfy our required properties for garbling schemes. For better concrete efficiency, we take an instantiation based on the BGV framework [BGV12, COS+22] in Appendix A.

**Definition 5** (SWHE)**.** *A somewhat homomorphic encryption scheme is a tuple of probabilistic polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ *for key space $\mathcal{K} \subseteq \mathcal{R}$, message space $\mathcal{M} \subseteq \mathcal{R}_p$, and ciphertext space $\mathcal{C}$.*

1. $\mathsf{params} \leftarrow \mathsf{Setup}(1^\lambda, 1^L)$. *The setup algorithm takes as input a security parameter $\lambda$ and a maximum multiplication depth $L$, and outputs a set of parameters* params*, which is an implicit input to the following algorithms and will be omitted for simplicity.*
2. $(\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params})$. *The key-generation algorithm takes as input a set of parameters* params*, and outputs a set of key-related parameters* keypar *and a secret key $\Delta \in \mathcal{K}$. Similarly,* keypar *is an implicit input to the following algorithms and will be omitted for simplicity.*
3. $[\![m]\!] \leftarrow \mathsf{Enc}(\Delta, m)$. *The encryption algorithm takes as input a secret key $\Delta \in \mathcal{K}$ and a plaintext $m \in \mathcal{M}$, and outputs a ciphertext $[\![m]\!]$.*
4. $m \leftarrow \mathsf{Dec}(\Delta, [\![m]\!])$. *The decryption algorithm takes as input a secret key $\Delta \in \mathcal{K}$ and a ciphertext $[\![m]\!] \in \mathcal{C}$, and outputs a plaintext $m \in \mathcal{M}$.*
5. $[\![y]\!] \leftarrow \mathsf{HEval}(f, [\![m_0]\!], \cdots, [\![m_{\ell-1}]\!])$. *The homomorphic evaluation algorithm takes as input a polynomial-sized arithmetic circuit $f : \mathcal{M}^\ell \to \mathcal{M}$ and $\ell$ ciphertexts $[\![m_0]\!], \cdots, [\![m_{\ell-1}]\!] \in \mathcal{C}$, and outputs a ciphertext $[\![y]\!] \in \mathcal{C}$.*

*For each security parameter $\lambda$, depth bound $L$, circuit $f : \mathcal{M}^\ell \to \mathcal{M}$ with multiplication depth at most $L$, and $m_0, \ldots, m_{\ell-1} \in \mathcal{M}$, it holds that*

$$\Pr\left[\mathsf{Dec}(\Delta, [\![y]\!]) \neq f(m_0, \cdots, m_{\ell-1}) \;\middle|\; \begin{array}{l} \mathsf{params} \leftarrow \mathsf{Setup}(1^\lambda, 1^L), \\ (\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params}), \\ [\![m_i]\!] \leftarrow \mathsf{Enc}(\Delta, m_i)\ \forall\, i \in [\ell], \\ [\![y]\!] \leftarrow \mathsf{HEval}(f, [\![m_0]\!], \cdots, [\![m_{\ell-1}]\!]) \end{array}\right] \leq \mathsf{negl}(\lambda)\,.$$

**Distributed decryption property.** The linear distributed decryption (DD) property (i.e., DD of dimension $k = 1$) was introduced by Boyle et al. [BKS19], and was extended to further support the correlated-key DD property (i.e., DD of dimension $k = 2$) by Liu et al. [LWYY25b]. We recall the DD property in the following definition, and also generalize the definition to support any DD dimension $k \in \mathbb{N}^+$, which may be useful for future garbling constructions. In this work, SWHEwDD refers to SWHE schemes that satisfy the following distributed decryption property on both $k = 1$ and $k = 2$.

**Definition 6** (Distributed Decryption)**.** *We say that an SWHE scheme satisfies the distributed decryption property on dimension $k \in \mathbb{N}^+$, if there exist polynomial-time algorithms $\mathsf{DD}_{\boldsymbol{a}} : \mathcal{R}_p^k \times \mathcal{C} \to \mathcal{R}_p$ for all $\boldsymbol{a} \in \{0,1\}^k$, each of which implicitly takes $(\mathsf{params}, \mathsf{keypar})$ as an input, such that, for all security parameter $\lambda$ and depth bound $L$, it holds that*

$$\Pr\left[\begin{array}{l} \\ \neq \end{array}\begin{array}{l} \mathsf{DD}_{\boldsymbol{x}}(\boldsymbol{X} + \boldsymbol{x}\cdot\Delta, \tau) \\ \mathsf{DD}_{\boldsymbol{0}}(\boldsymbol{X}, \tau) + \left(\prod_{i=0}^{k-1} x_i\right)\cdot \mathsf{Dec}(\Delta, \tau) \end{array}\middle|\begin{array}{l} \mathsf{params} \leftarrow \mathsf{Setup}(1^\lambda, 1^L), \\ (\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params}), \\ \boldsymbol{X} \xleftarrow{\$} \mathcal{R}_p^k, \\ \forall \tau \in \mathcal{C}, \boldsymbol{x} \in \{0,1\}^k \end{array}\right] \leq \mathsf{negl}(\lambda)\ ,$$

*where the operation of $\boldsymbol{X} + \boldsymbol{x}\cdot\Delta$ is defined over $\mathcal{R}_p^k$.*

That is, if an SWHE scheme satisfies DD on $k = 1$, with overwhelming probability it holds that $\mathsf{DD}_i(X + i\cdot\Delta, \tau) = \mathsf{DD}_0(X, \tau) + i\cdot\mathsf{Dec}(\Delta, \tau)$ for any bit $i \in \{0,1\}$ and uniformly distributed $X \xleftarrow{\$} \mathcal{R}_p$; if one satisfies DD on $k = 2$, with overwhelming probability it holds that $\mathsf{DD}_{i,j}(X + i\cdot\Delta, Y + j\cdot\Delta, \tau) = \mathsf{DD}_{0,0}(X, Y, \tau) + i\cdot j\cdot\mathsf{Dec}(\Delta, \tau)$ for any $i, j \in \{0,1\}$ and uniformly distributed $X, Y \xleftarrow{\$} \mathcal{R}_p$.

To simplify the notation of our garbling scheme, for each $i, j \in \{0,1\}$, we define the function $\mathsf{Eval}_{i,j} : \mathcal{R}_p^2 \times \mathcal{C}^3 \to \mathcal{R}_p$ as

$$\mathsf{Eval}_{i,j}(A, B, \tau_1, \tau_2, \tau_3) := \mathsf{DD}_i(A, \tau_1) + \mathsf{DD}_j(B, \tau_2) + \mathsf{DD}_{i,j}(A, B, \tau_3)\ .$$

**CPA and KDM security properties.** In our garbling schemes, we require that the SWHEwDD scheme satisfies the standard indistinguishability under chosen-plaintext attacks (i.e., IND-CPA in short). Informally, any PPT adversary cannot distinguish $[\![m_0]\!]$ from $[\![m_1]\!]$ for two adversarial-chosen messages $m_0, m_1$, given that the adversary obtains $(\mathsf{params}, \mathsf{keypar})$ and has access to an encryption oracle $\mathsf{Enc}(\Delta, \cdot)$. Furthermore, we assume that the SWHEwDD scheme also satisfies the key-dependent message (KDM) security [BRS03]. Informally, for any PPT adversary $\mathcal{A}$, it is infeasible that $\mathcal{A}$ can distinguish $[\![f(\Delta)]\!]$ from $[\![0]\!]$ for a function $f$ chosen by itself, given that $\mathcal{A}$ has access to $(\mathsf{params}, \mathsf{keypar})$ as well as $\mathsf{Enc}(\Delta, \cdot)$. For our garbling application, we only need that $[\![\Delta]\!]$ is computationally indistinguishable from $[\![0]\!]$.

# 4 BitGC-small: Garbling Scheme with 1 Bit per Gate

## 4.1 Detailed Construction of BitGC-small

The details of garbling scheme BitGC-small are shown in Figures 4 and 5. Compared to the original BitGC scheme [LWYY25b], BitGC-small achieves the same communication cost but has a much smaller computational cost. The computation is improved by supporting PRG homomorphic evaluation with SIMD optimization and applying specific optimizations on XOR gates. In particular, for each gate $(\alpha, \beta, \gamma, \mathsf{XOR})$, the masked bit on the output wire $\gamma$ is equal to

$$\hat{x}_\gamma = (x_\alpha \oplus x_\beta) \oplus \pi_\gamma = (\hat{x}_\alpha \oplus \pi_\alpha) \oplus (\hat{x}_\beta \oplus \pi_\beta) \oplus \pi_\gamma = \hat{x}_\alpha \oplus \hat{x}_\beta \oplus (\pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma)\ .$$

Thus, we let the garbler send $d_\gamma = \pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma$ to the evaluator, who can compute the masked bit $\hat{x}_\gamma = \hat{x}_\alpha \oplus \hat{x}_\beta \oplus d_\gamma$. For each XOR gate with output wire $\gamma$, we have $d_\gamma = \hat{z}^{0,0}$ according to their definitions. Furthermore, the relationships $\hat{z}^{1,0} = \hat{z}^{0,1} = \hat{z}^{0,0} \oplus 1 = 1 - \hat{z}^{0,0}$ and $\hat{z}^{1,1} = \hat{z}^{0,0}$ hold for each XOR gate. Therefore, the ciphertexts $\tau_1, \tau_2, \tau_3$ can be computed in a simple manner without any homomorphic multiplications.

The BitGC-small adopts a low-depth PRG, which can be instantiated under different assumptions [Gol11, Ale03, App12, BPR12, AL16, AR16, ABBS25]. Our implementation uses a Goldreich-style random local

BitGC-small (Part I)

Let $\mathsf{SWHE} = (\mathsf{Setup}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ be an SWHEwDD scheme. Let $\mathsf{PRG} : \{0,1\}^{\ell} \to \{0,1\}^{N}$ be a secure $t$-depth PRG where $\ell$ is the seed size and $N = |\mathcal{I}| + |\mathcal{W}|$ is the number of wires, $\mathsf{PRF} : \{0,1\}^{\lambda} \times \{0,1\}^{\lambda} \to \mathcal{R}_p$ be a secure PRF, and $\mathsf{H} : \mathcal{R}_p \to \{0,1\}^{\lambda}$ be a random oracle.

$\mathsf{Gb}(1^{\lambda}, f)$**:**

1. Generate SWHE parameters $\mathsf{params} \leftarrow \mathsf{Setup}(1^{\lambda}, 1^{L})$, where $L = t + 2$ is the maximum multiplication depth for SWHE.
2. Produce a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params})$ where $\mathsf{LSB}(\Delta) = 1$, and generate $[\![\Delta]\!] := \mathsf{Enc}(\Delta, \Delta)$. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^{\lambda}$.
3. Sample a PRG seed $\boldsymbol{s} \xleftarrow{\$} \{0,1\}^{\ell}$ and compute ciphertext $[\![s_i]\!] := \mathsf{Enc}(\Delta, s_i)$ for each $i \in [\ell]$. Then, compute $(\pi_0, \ldots, \pi_{N-1}) := \mathsf{PRG}(\boldsymbol{s})$, and homomorphically compute ciphertexts $([\![\pi_0]\!], \ldots, [\![\pi_{N-1}]\!]) := \mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$.
4. For each input wire $i \in \mathcal{I}$, sample a label $W_i^0 \xleftarrow{\$} \mathcal{R}_p$ and compute $W_i^1 := W_i^0 + (-1)^{\pi_i} \cdot \Delta \in \mathcal{R}_p$.
5. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, compute a bit $\hat{z}^{i,j} := (\pi_\alpha \oplus i) \wedge (\pi_\beta \oplus j) \oplus \pi_\gamma$ and homomorphically compute a ciphertext $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!], [\![\pi_\beta]\!], [\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$ with two levels of homomorphic multiplications.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!], \tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!], \tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute a pair of labels $W_\gamma^{\pi_\gamma} := \mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) - \hat{z}^{0,0} \cdot \Delta + \mathsf{PRF}_K(\gamma)$ and $W_\gamma^{\pi_\gamma \oplus 1} := W_\gamma^{\pi_\gamma} + \Delta$, along with a bit $d_\gamma := \mathsf{LSB}(W_\gamma^0) \oplus \pi_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute a bit $d_\gamma := \pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma$ as well as three ciphertexts $\tau_1 := (1 - 2 \cdot d_\gamma) \cdot [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (b) Compute a pair of labels $W_\gamma^{\pi_\gamma} := \mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) - d_\gamma \cdot \Delta + \mathsf{PRF}_K(\gamma)$ and $W_\gamma^{\pi_\gamma \oplus 1} := W_\gamma^{\pi_\gamma} + \Delta$.
6. Output a garbled circuit $F := (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{[\![s_i]\!]\}_{i \in [\ell]}, \{d_i\}_{i \in \mathcal{W}})$, an encoding information $e := (\{(\pi_i, W_i^0)\}_{i \in \mathcal{I}}, \Delta)$ and a decoding information $d := \left\{\left(\mathsf{H}(W_i^0), \mathsf{H}(W_i^1)\right)\right\}_{i \in \mathcal{O}}$.

$\mathsf{En}(e, x)$**:** For each $i \in \mathcal{I}$, compute $\hat{x}_i := x_i \oplus \pi_i$ and $W_i := W_i^0 + (-1)^{\pi_i} \cdot x_i \cdot \Delta \in \mathcal{R}_p$. Output $X := \{(\hat{x}_i, W_i)\}_{i \in \mathcal{I}}$.

Figure 4: Garbling scheme BitGC-small with 1 bit per gate (Part I).

PRG [AL16, ABBS25], which is able to expand a $\ell$-bit seed into $\ell^2$ pseudorandom bits using a depth $t = 4$ arithmetic circuit. See Section 6 for more details. In this case, BitGC-small requires SWHEwDD to support $t + 2 = 6$ levels of multiplications. We use a pseudorandom function (PRF) to randomize the labels on the output wires of all gates. This makes the DD property be satisfied in the computational sense. Note that PRF is not used to protect the privacy, and instead guarantees the correctness. For a secret key $\Delta$, we assume that $\mathsf{LSB}(\Delta) = 1$. In this case, since $W_i = W_i^0 + (-1)^{\pi_i} \cdot x_i \cdot \Delta = W_i^{\pi_i} + (x_i \oplus \pi_i) \cdot \Delta$ due to the fact that $W_i^{\pi_i} = W_i^0 + (-1)^{\pi_i} \cdot \pi_i \cdot \Delta$ and $(-1)^{\pi_i} \cdot (x_i - \pi_i) = x_i \oplus \pi_i$, we have $\mathsf{LSB}(W_i) = \mathsf{LSB}(W_i^{\pi_i}) \oplus (x_i \oplus \pi_i)$ and $\mathsf{LSB}(W_i^{\pi_i}) = \mathsf{LSB}(W_i^0) \oplus \pi_i$.

The BitGC-small scheme satisfies not only privacy but also obliviousness and authenticity. To achieve authenticity, we use the hash values of a pair of labels for each circuit-output wire as the decoding information, following the standard approach used in prior works [HKE12, ZRE15, LWYY25b]. In proof of security, we model the hash function as a random oracle (RO). In the following subsections, we analyze the correctness of garbling scheme BitGC-small and prove its security. If authenticity is not required, then we can set the

BitGC-small (Part II)

$\mathsf{Ev}(F, X)$:

1. Homomorphically compute $([\![\pi_0]\!], \ldots, [\![\pi_{N-1}]\!]) := \mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$.
2. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, perform the following steps:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0, 1\}$, homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!]$, $[\![\pi_\beta]\!]$, $[\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, and $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Compute a masked bit $\hat{x}_\gamma := \mathsf{LSB}(W_\gamma) \oplus d_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute $\tau_1 := (1 - 2 \cdot d_\gamma) \cdot [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (b) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (c) Compute a masked bit $\hat{x}_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta \oplus d_\gamma$.
3. Output $Y := \{W_i\}_{i \in \mathcal{O}}$.

$\mathsf{De}(d, Y)$: For each $i \in \mathcal{O}$, if $\mathsf{H}(W_i) = \mathsf{H}(W_i^0)$, set $x_i := 0$; if $\mathsf{H}(W_i) = \mathsf{H}(W_i^1)$, set $x_i := 1$; otherwise, abort and output $\perp$. If not abort, output $y := \{x_i\}_{i \in \mathcal{O}}$.

Figure 5: Garbling scheme BitGC-small with 1 bit per gate (Part II).

permutation bit $\pi_i$ for each $i \in \mathcal{O}$ as the decoding information; the evaluation algorithm outputs the masked bit $\hat{x}_i$, and the decoding algorithm recovers the output bit $x_i = \hat{x}_i \oplus \pi_i$, for each $i \in \mathcal{O}$. In addition, we note that one can replace the random oracle model with the notion of correlation-robust hash functions (CRHFs) [IKNP03, BHKR13, GKWY20, GKW+20, GWYY25] to remove the dependency on the RO and achieve better computational efficiency. In particular, while the domain of a CRHF $\mathsf{H}$ is often defined over $\{0, 1\}^\lambda$, we need to extend the domain of $\mathsf{H}$ to $\mathcal{R}_p$ for our garbling schemes. The standard techniques as described above can be applied in not only BitGC-small but also BitGC-fast and BitGC-tiny shown in Section 5.

**Analysis for communication complexity.** We focus on the size of a garbled circuit $F$, as it dominates the communication cost. We can always suppose that the garbler and evaluator have already agreed upon the set of parameters params. The keypar and $[\![\Delta]\!]$ can be reused across multiple garbling executions between the garbler and evaluator, which allows us to amortize the communication cost to negligibly small. Furthermore, the communication cost of transmitting keypar and $[\![\Delta]\!]$ is independent of the circuit size. Excluding the tuple of system parameters $(\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!])$, the size of garbled circuit $F$ is $|\mathcal{W}| + \ell \cdot |ct| + \lambda$ bits, where $|\mathcal{W}|$ is the number of gates, $\ell = o(|\mathcal{W}|)$ is the seed size of the PRG, and $|ct|$ is the size of an SWHE ciphertext. Therefore, the amortized communication cost for each gate is only 1 bit for sufficiently large circuits.

## 4.2 Correctness Analysis

In the following, we analyze the correctness of the BitGC-small scheme (Figure 4 and 5). We replace $\mathsf{PRF}_K(\gamma)$ with a uniform element $R_\gamma \in \mathcal{R}_p$ for each gate-output wire $\gamma \in \mathcal{W}$. This replacement only incurs a negligible difference for the correctness probability, due to the pseudorandomness of PRF outputs, following the correctness analysis in prior works [BKS19, LWYY25b]. In this case, we have that $W_\gamma^{\pi_\gamma}$ is uniform in $\mathcal{R}_p$ for each $\gamma \in \mathcal{W}$. For each input wire $i \in \mathcal{I}$, we have $W_i^{\pi_i} = W_i^0 + (-1)^{\pi_i} \cdot \pi_i \cdot \Delta$ is uniform in $\mathcal{R}_p$, due to the uniformity of $W_i^0$.

We prove the correctness by induction following the topological order of the circuit. Throughout this

paper, for each wire $i$, we use $x_i$ and $\hat{x}_i$ to denote the real value and masked value, respectively. For each input wire $i \in \mathcal{I}$, we have that $W_i = W_i^0 + (-1)^{\pi_i} \cdot x_i \cdot \Delta$ and $\hat{x}_i = x_i \oplus \pi_i$. For each gate $(\alpha, \beta, \gamma, T)$, we have the following holds:

$$W_\alpha = W_\alpha^0 + (-1)^{\pi_\alpha} \cdot x_\alpha \cdot \Delta \text{ and } \hat{x}_\alpha = x_\alpha \oplus \pi_\alpha,$$
$$W_\beta = W_\beta^0 + (-1)^{\pi_\beta} \cdot x_\beta \cdot \Delta \text{ and } \hat{x}_\beta = x_\beta \oplus \pi_\beta\,.$$

According to the definitions of $(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta})$ and $(\hat{x}_\alpha, \hat{x}_\beta)$, it is easy to verify that $W_\alpha = W_\alpha^{\pi_\alpha} + \hat{x}_\alpha \cdot \Delta$ and $W_\beta = W_\beta^{\pi_\beta} + \hat{x}_\beta \cdot \Delta$. For the output wire $\gamma$, we show that $W_\gamma = W_\gamma^{\pi_\gamma} + \hat{x}_\gamma \cdot \Delta$ and $\hat{x}_\gamma = \pi_\gamma \oplus x_\gamma$ hold, except with negligible probability. We consider two cases: $T = \mathsf{AND}$ and $T = \mathsf{XOR}$.

**Case 1:** $T = \mathsf{AND}$**.** In the $\mathsf{Gb}$ algorithm, for the output wire $\gamma$, $W_\gamma^{\pi_\gamma}$ is computed as $\mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) - \hat{z}^{0,0} \cdot \Delta + R_\gamma$ where $\hat{z}^{0,0} = (\pi_\alpha \wedge \pi_\beta) \oplus \pi_\gamma$. Then, $W_\gamma^0$ is computed as $W_\gamma^{\pi_\gamma} - (-1)^{\pi_\gamma} \cdot \pi_\gamma \cdot \Delta$. In the $\mathsf{Ev}$ algorithm, we have that $W_\gamma = \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + R_\gamma$ and $\hat{x}_\gamma = \mathsf{LSB}(W_\gamma) \oplus d_\gamma$, where $d_\gamma = \mathsf{LSB}(W_\gamma^0) \oplus \pi_\gamma$. Based on the distributed decryption property, with overwhelming probability, we have

$$\begin{aligned}
W_\gamma &= \mathsf{DD}_{\hat{x}_\alpha}(W_\alpha, \tau_1) + \mathsf{DD}_{\hat{x}_\beta}(W_\beta, \tau_2) + \mathsf{DD}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_3) + R_\gamma \\
&= \mathsf{DD}_{\hat{x}_\alpha}(W_\alpha^{\pi_\alpha} + \hat{x}_\alpha \cdot \Delta, \tau_1) + \mathsf{DD}_{\hat{x}_\beta}(W_\beta^{\pi_\beta} + \hat{x}_\beta \cdot \Delta, \tau_2) \\
&\quad + \mathsf{DD}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha^{\pi_\alpha} + \hat{x}_\alpha \cdot \Delta, W_\beta^{\pi_\beta} + \hat{x}_\beta \cdot \Delta, \tau_3) + R_\gamma \\
&= \mathsf{DD}_0(W_\alpha^{\pi_\alpha}, \tau_1) + \mathsf{DD}_0(W_\beta^{\pi_\beta}, \tau_2) + \mathsf{DD}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_3) + R_\gamma \\
&\quad + \hat{x}_\alpha \cdot \mathsf{Dec}(\Delta, \tau_1) + \hat{x}_\beta \cdot \mathsf{Dec}(\Delta, \tau_2) + \hat{x}_\alpha \cdot \hat{x}_\beta \cdot \mathsf{Dec}(\Delta, \tau_3) \\
&= \mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) + R_\gamma + \big(\hat{x}_\alpha \cdot (\hat{z}^{1,0} - \hat{z}^{0,0}) + \\
&\quad \hat{x}_\beta \cdot (\hat{z}^{0,1} - \hat{z}^{0,0}) + \hat{x}_\alpha \cdot \hat{x}_\beta \cdot (\hat{z}^{1,1} + \hat{z}^{0,0} - \hat{z}^{1,0} - \hat{z}^{0,1})\big) \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + \hat{z}^{0,0} \cdot \Delta + (\hat{z}^{\hat{x}_\alpha, \hat{x}_\beta} - \hat{z}^{0,0}) \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + \hat{z}^{0,0} \cdot \Delta + (\hat{x}_\gamma - \hat{z}^{0,0}) \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + \hat{x}_\gamma \cdot \Delta = W_\gamma^{\pi_\gamma} + (x_\gamma \oplus \pi_\gamma) \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + (-1)^{\pi_\gamma} \cdot (x_\gamma - \pi_\gamma) \cdot \Delta \\
&= W_\gamma^0 + (-1)^{\pi_\gamma} \cdot x_\gamma \cdot \Delta\,.
\end{aligned} \tag{4}$$

In the above analysis, since $\hat{z}^{i,j} = (\pi_\alpha \oplus i) \wedge (\pi_\beta \oplus j) \oplus \pi_\gamma$, $\hat{x}_\alpha = x_\alpha \oplus \pi_\alpha$ and $\hat{x}_\beta = x_\beta \oplus \pi_\beta$, we obtain that $\hat{z}^{\hat{x}_\alpha, \hat{x}_\beta} = (x_\alpha \wedge x_\beta) \oplus \pi_\gamma = x_\gamma \oplus \pi_\gamma = \hat{x}_\gamma$. In addition, we use the fact that $x_\gamma \oplus \pi_\gamma = (-1)^{\pi_\gamma} \cdot (x_\gamma - \pi_\gamma)$ for $x_\gamma, \pi_\gamma \in \{0, 1\}$. Thus, we have

$$\hat{x}_\gamma = \mathsf{LSB}(W_\gamma) \oplus d_\gamma = \mathsf{LSB}(W_\gamma) \oplus \mathsf{LSB}(W_\gamma^0) \oplus \pi_\gamma = x_\gamma \oplus \pi_\gamma\,.$$

**Case 2:** $T = \mathsf{XOR}$**.** In the $\mathsf{Gb}$ algorithm, for the output wire $\gamma$, we still have the equations $W_\gamma^{\pi_\gamma} = \mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) - d_\gamma \cdot \Delta + R_\gamma$ and $W_\gamma^0 = W_\gamma^{\pi_\gamma} - (-1)^{\pi_\gamma} \cdot \pi_\gamma \cdot \Delta$, where $d_\gamma = \pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma$. In the $\mathsf{Ev}$ algorithm, we still have $W_\gamma = \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + R_\gamma$. Differently, the masked bit $\hat{x}_\gamma$ is computed as $\hat{x}_\alpha \oplus \hat{x}_\beta \oplus d_\gamma$. From the distributed decryption property, except with negligible probability,

$$\begin{aligned}
W_\gamma &= W_\gamma^{\pi_\gamma} + d_\gamma \cdot \Delta + \hat{x}_\alpha \cdot \mathsf{Dec}(\Delta, \tau_1) + \hat{x}_\beta \cdot \mathsf{Dec}(\Delta, \tau_2) + \hat{x}_\alpha \cdot \hat{x}_\beta \cdot \mathsf{Dec}(\Delta, \tau_3) \\
&= W_\gamma^{\pi_\gamma} + d_\gamma \cdot \Delta + \big(\hat{x}_\alpha + \hat{x}_\beta - 2 \cdot \hat{x}_\alpha \cdot \hat{x}_\beta\big) \cdot (-1)^{d_\gamma} \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + d_\gamma \cdot \Delta + \big(\hat{x}_\alpha \oplus \hat{x}_\beta\big) \cdot (-1)^{d_\gamma} \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + d_\gamma \cdot \Delta + (\hat{x}_\alpha \oplus \hat{x}_\beta \oplus d_\gamma - d_\gamma) \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + (x_\alpha \oplus x_\beta \oplus \pi_\gamma) \cdot \Delta \\
&= W_\gamma^{\pi_\gamma} + \hat{x}_\gamma \cdot \Delta \\
&= W_\gamma^0 + (-1)^{\pi_\gamma} \cdot x_\gamma \cdot \Delta\,.
\end{aligned} \tag{5}$$

In the above analysis, we omit some intermediate steps, as they can be derived by applying the same approach as in Equation (4). Note that $\tau_1$, $\tau_2$ and $\tau_3$ are now decrypted as

$$\mathsf{Dec}(\Delta, \tau_1) = \mathsf{Dec}(\Delta, \tau_2) = (1 - 2 \cdot d_\gamma) \cdot \Delta = (-1)^{d_\gamma} \cdot \Delta$$
$$\mathsf{Dec}(\Delta, \tau_3) = -2 \cdot (-1)^{d_\gamma} \cdot \Delta\,,$$

Next, it is easy to verify

$$\hat{x}_\gamma = \hat{x}_\alpha \oplus \hat{x}_\beta \oplus d_\gamma = \hat{x}_\alpha \oplus \hat{x}_\beta \oplus \pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma = x_\alpha \oplus x_\beta \oplus \pi_\gamma = x_\gamma \oplus \pi_\gamma\,.$$

From the above analysis, for each output wire $i \in \mathcal{O}$, we have $W_i = W_i^0 + (-1)^{\pi_i} \cdot x_i \cdot \Delta$ where $\{x_i\}_{i\in\mathcal{O}} = f(\{x_i\}_{i\in\mathcal{I}})$. Recall that $\mathsf{De}(d, Y)$ outputs 0 if $\mathsf{H}(W_i) = \mathsf{H}(W_i^0)$, and outputs 1 if $\mathsf{H}(W_i) = \mathsf{H}(W_i^1)$. Therefore, $\mathsf{De}(d, Y)$ outputs $y = f(x)$, which completes the correctness proof.

### 4.3 Security Proof

Before giving the detailed proof, we show an overview of the proof. We complete the proof via a sequence of hybrids, where the real-world execution is **Hybrid 0**. In **Hybrid 1**, the Gb algorithm only computes the labels on the real bits without knowing $\Delta$, following the computation process on the evaluator side. Furthermore, each bit $d_\gamma$ is computed with the masked bit and $\mathsf{LSB}(W_\gamma)$, given that each masked bit can be computed with the real bit and permutation bit. These changes are computationally indistinguishable from the real-world execution, according to the correctness analysis based on the pseudorandomness of the PRF outputs. In this hybrid, the decoding information is set according to the circuit output $y = f(x)$, and $\mathsf{H}(W_i^{y_i\oplus 1})$ is replaced with a random string $V_i$. In the random oracle (RO) model, if a PPT adversary can distinguish the decoding information from real values, then we can use it to break the IND-CPA and KDM security of the SWHEwDD scheme. This hybrid allows us to generate a garbled circuit without depending on the secret key $\Delta$, which is crucial for going into the next hybrid. In **Hybrid 2**, we replace $\mathsf{Enc}(\Delta, \Delta)$ and $\mathsf{Enc}(\Delta, s_i)$ for $i \in [\ell]$ with the ciphertexts on zero. It is easy to see that Hybrid 2 is computationally indistinguishable from Hybrid 1, under the assumption that the SWHEwDD scheme is IND-CPA and KDM secure. In **Hybrid 3**, we replace each permutation bit $\pi_i$ with a random bit, which has a negligible difference based on the pseudorandomness of the PRG output. In **Hybrid 4**, we replace each bit $d_\gamma$ with a random bit. This hybrid is perfectly indistinguishable from Hybrid 3, due to the fact that $\pi_\gamma$ is uniform. In Hybrid 4, no real bits are used, and we construct a PPT simulator for BitGC-small. That is, Hybrid 4 is the ideal-world execution. The above overview is used for the proof of privacy. The proof of obliviousness can use the same simulator and hybrids, except that the decoding information is not sent to the adversary, and so need not be simulated. For authenticity, we can use the simulator of privacy to generate the garbled circuit $F$ and input labels. If a PPT adversary can break authenticity, then it must make a query associated with $\Delta$ to the random oracle H. We can extract $\Delta$ from the queries of H and use it to break the IND-CPA and KDM security of the SWHEwDD scheme.

**Theorem 1.** *Garbling scheme* BitGC-small *shown in Figures 4 and 5 satisfies privacy, obliviousness and authenticity, provided that the SWHEwDD scheme satisfies the IND-CPA and KDM security properties,* PRG *is a secure PRG,* PRF *is a secure PRF, and* H *is a random oracle.*

The detailed proof of Theorem 1 can be found in Appendix B.1.

## 5 BitGC-fast and BitGC-tiny: Garbling Schemes with Free-XOR

### 5.1 BitGC-fast with Free-XOR and 5 Bits per AND Gate

The detailed construction of the BitGC-fast garbling scheme is shown in Figures 6 and 7. The BitGC-fast scheme adopts the En and De algorithms in the BitGC-small scheme (Section 4.1). Thus, we omit the description of

BitGC-fast (Part I)

Let $\mathsf{SWHE} = (\mathsf{Setup}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ be an SWHE scheme with distributed decryption. Let $\mathsf{PRG} : \{0,1\}^{\ell} \to \{0,1\}^{4N}$ be a secure $t$-depth PRG where $\ell$ is the seed size and $N = |\mathcal{W}_{\mathsf{AND}}|$ is the number of AND gates, $\mathsf{PRF} : \{0,1\}^{\lambda} \times \{0,1\}^{\lambda} \to \mathcal{R}_p$ be a secure PRF, and $\mathsf{H} : \mathcal{R}_p \to \{0,1\}^{\lambda}$ be a random oracle.

$\mathsf{Gb}(1^{\lambda}, f)$:

1. Generate a set of setup parameters $\mathsf{params} \leftarrow \mathsf{Setup}(1^{\lambda}, 1^{L})$, where $L = t + 1$ is the maximum multiplication depth for SWHEwDD.
2. Generate a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params})$ with $\mathsf{LSB}(\Delta) = 1$, and run $[\![\Delta]\!] := \mathsf{Enc}(\Delta, \Delta)$. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^{\lambda}$.
3. Sample a PRG seed $\boldsymbol{s} \xleftarrow{\$} \{0,1\}^{\ell}$ and generate $[\![s_i]\!] := \mathsf{Enc}(\Delta, s_i)$ for each bit $s_i$ with $i \in [\ell]$. Then, compute $\{r_{\gamma}^{i,j}\}_{\gamma \in \mathcal{W}_{\mathsf{AND}}, i,j \in \{0,1\}} := \mathsf{PRG}(\boldsymbol{s})$, and homomorphically compute $\mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$ to obtain $[\![r_{\gamma}^{i,j}]\!]$ for each $\gamma \in \mathcal{W}_{\mathsf{AND}}, i, j \in \{0,1\}$.
4. For each $\gamma \in \mathcal{W}_{\mathsf{AND}}, i, j \in \{0,1\}$, homomorphically compute a ciphertext $[\![r_{\gamma}^{i,j} \cdot \Delta]\!]$ from $[\![r_{\gamma}^{i,j}]\!]$ and $[\![\Delta]\!]$.
5. For each input wire $i \in \mathcal{I}$, sample $\pi_i \xleftarrow{\$} \{0,1\}$ and $W_i^0 \xleftarrow{\$} \mathcal{R}_p$, and then compute $W_i^1 := W_i^0 + (-1)^{\pi_i} \cdot \Delta \in \mathcal{R}_p$.
6. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) Sample $\pi_\gamma \xleftarrow{\$} \{0,1\}$. For each $i, j \in \{0,1\}$, compute $\hat{z}^{i,j} := (\pi_\alpha \oplus i) \wedge (\pi_\beta \oplus j) \oplus \pi_\gamma$, set $d_{\gamma}^{i,j} := \hat{z}^{i,j} \oplus r_{\gamma}^{i,j}$, and homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!] := d_{\gamma}^{i,j} \cdot [\![\Delta]\!] + (1 - 2 \cdot d_{\gamma}^{i,j}) \cdot [\![r_{\gamma}^{i,j} \cdot \Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute a pair of labels $W_{\gamma}^{\pi_\gamma} := \mathsf{Eval}_{0,0}(W_{\alpha}^{\pi_\alpha}, W_{\beta}^{\pi_\beta}, \tau_1, \tau_2, \tau_3) - \hat{z}^{0,0} \cdot \Delta + \mathsf{PRF}_K(\gamma)$, $W_{\gamma}^{\pi_\gamma \oplus 1} := W_{\gamma}^{\pi_\gamma} + \Delta$, along with a bit $d_\gamma := \mathsf{LSB}(W_{\gamma}^{0}) \oplus \pi_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute $\pi_\gamma := \pi_\alpha \oplus \pi_\beta$, $\tau_1 := [\![\Delta]\!]$, $\tau_2 := \tau_1$, and $\tau_3 := -\tau_1 - \tau_2$.
     (b) Compute a pair of labels $W_{\gamma}^{\pi_\gamma} := \mathsf{Eval}_{0,0}(W_{\alpha}^{\pi_\alpha}, W_{\beta}^{\pi_\beta}, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$ and $W_{\gamma}^{\pi_\gamma \oplus 1} := W_{\gamma}^{\pi_\gamma} + \Delta$.
7. Output a garbled circuit $F := (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{[\![s_i]\!]\}_{i \in [\ell]}, \{(d_\gamma, d_{\gamma}^{0,0}, d_{\gamma}^{0,1}, d_{\gamma}^{1,0}, d_{\gamma}^{1,1})\}_{\gamma \in \mathcal{W}_{\mathsf{AND}}})$, an encoding information $e := \big(\{(\pi_i, W_i^0)\}_{i \in \mathcal{I}}, \Delta\big)$ and a decoding information $d := \{\mathsf{H}(W_i^0), \mathsf{H}(W_i^1)\}_{i \in \mathcal{O}}$.

Figure 6: BitGC-fast with 5 bits per AND gate and free-XOR (Part I), where the differences between BitGC-fast and BitGC-small are marked in blue.

the two algorithms for BitGC-fast. In Figures 6 and 7, the steps marked in blue show the differences of the Gb and Ev algorithms between BitGC-fast and BitGC-small. The main differences are listed as follows:

- Homomorphic PRG computation needs to generate the SWHE ciphertexts on $4 \cdot |\mathcal{W}_{\mathsf{AND}}|$ (instead of $|\mathcal{I}| + |\mathcal{W}|$) pseudorandom bits
- Each bit $\hat{z}^{i,j}$ is masked with a pseudorandom bit $r_{\gamma}^{i,j}$ and sent to the evaluator. Then the evaluator can compute the ciphertext $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ with only homomorphic-addition operations.
- For the output wire $\gamma$ of each AND gate, the permutation bit $\pi_\gamma$ is sampled uniformly rather than an output bit of $\mathsf{PRG}(\boldsymbol{s})$. For each XOR gate, $\pi_\gamma$ is defined as $\pi_\alpha \oplus \pi_\beta$. In this case, the bit $d_\gamma = \pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma = 0$ need not be sent, achieving free communication for XOR gates.

In the amortized setting, BitGC-fast takes 5 bits per AND gate and achieves free-XOR. It requires that the SWHEwDD scheme supports $t + 1$ levels of homomorphic multiplications, compared with $t + 2$ levels for BitGC-small.

**Correctness analysis.** Compared to BitGC-small, the BitGC-fast scheme mainly changes how the bits $\{d_\gamma\}$

BitGC-fast (Part II)

$\mathsf{Ev}(F, X)$:

1. Homomorphically compute $\mathsf{PRG}(\llbracket s_0 \rrbracket, \ldots, \llbracket s_{\ell-1} \rrbracket)$ to obtain $\llbracket r_\gamma^{i,j} \rrbracket$ for each $\gamma \in \mathcal{W}_{\mathsf{AND}}, i, j \in \{0,1\}$. Then, for each $\gamma \in \mathcal{W}_{\mathsf{AND}}, i, j \in \{0,1\}$, homomorphically compute $\llbracket r_\gamma^{i,j} \cdot \Delta \rrbracket$ from $\llbracket r_\gamma^{i,j} \rrbracket$ and $\llbracket \Delta \rrbracket$.
2. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, perform the following steps:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, homomorphically compute $\llbracket \hat{z}^{i,j} \cdot \Delta \rrbracket := d_\gamma^{i,j} \cdot \llbracket \Delta \rrbracket + (1 - 2 \cdot d_\gamma^{i,j}) \cdot \llbracket r_\gamma^{i,j} \cdot \Delta \rrbracket$.
     (b) Compute $\tau_1 := \llbracket \hat{z}^{1,0} \cdot \Delta \rrbracket - \llbracket \hat{z}^{0,0} \cdot \Delta \rrbracket, \tau_2 := \llbracket \hat{z}^{0,1} \cdot \Delta \rrbracket - \llbracket \hat{z}^{0,0} \cdot \Delta \rrbracket, \tau_3 := \llbracket \hat{z}^{1,1} \cdot \Delta \rrbracket - \llbracket \hat{z}^{0,0} \cdot \Delta \rrbracket - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Compute a masked bit $\hat{x}_\gamma := \mathsf{LSB}(W_\gamma) \oplus d_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Set $\tau_1 := \llbracket \Delta \rrbracket$, $\tau_2 := \tau_1$, and $\tau_3 := -\tau_1 - \tau_2$.
     (b) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (c) Compute a masked bit $\hat{x}_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta$.
3. Output $Y := \{W_i\}_{i \in \mathcal{O}}$.

Figure 7: BitGC-fast with 5 bits per AND gate and free-XOR (Part II), where the differences between BitGC-fast and BitGC-small are marked in blue.

are computed. Specifically, for each AND gate, BitGC-fast uses four additional bits of communication to directly transmit $\{\hat{z}^{i,j}\}$ in a secure way. For each XOR gate, BitGC-fast directly sets $d_\gamma = 0$, which realizes free-XOR. Below, we show that these changes do not affect the correctness.

- **Case 1: each AND gate.** The BitGC-fast scheme derives $\tau_1$, $\tau_2$, and $\tau_3$ in the same way as BitGC-small, except that it uses a different method to generate $\llbracket \hat{z}^{i,j} \cdot \Delta \rrbracket$. Each term $\llbracket \hat{z}^{i,j} \cdot \Delta \rrbracket$ is homomorphically computed as

$$\llbracket \hat{z}^{i,j} \cdot \Delta \rrbracket := d_\gamma^{i,j} \cdot \llbracket \Delta \rrbracket + (1 - 2 \cdot d_\gamma^{i,j}) \cdot \llbracket r_\gamma^{i,j} \cdot \Delta \rrbracket .$$

  Therefore, it suffices to show that $\mathsf{Dec}(\llbracket \hat{z}^{i,j} \cdot \Delta \rrbracket) = \hat{z}^{i,j} \cdot \Delta$. In particular,

$$\begin{aligned} \mathsf{Dec}(\llbracket \hat{z}^{i,j} \cdot \Delta \rrbracket) &= d_\gamma^{i,j} \cdot \Delta + (1 - 2 \cdot d_\gamma^{i,j}) \cdot r_\gamma^{i,j} \cdot \Delta \\ &= \left(d_\gamma^{i,j} + r_\gamma^{i,j} - 2 \cdot d_\gamma^{i,j} \cdot r_\gamma^{i,j}\right) \cdot \Delta \\ &= \left(d_\gamma^{i,j} \oplus r_\gamma^{i,j}\right) \cdot \Delta = \hat{z}^{i,j} \cdot \Delta . \end{aligned}$$

- **Case 2: each XOR gate.** The BitGC-fast scheme directly defines $\pi_\gamma = \pi_\alpha \oplus \pi_\beta$, resulting in $d_\gamma = 0$. This modification preserves the correctness of the wire label, since the proof of Equation (5) remains valid. Therefore, it suffices to show that $\hat{x}_\gamma = \pi_\gamma \oplus x_\gamma$. To see this, observe that

$$\hat{x}_\gamma = \hat{x}_\alpha \oplus \hat{x}_\beta = x_\alpha \oplus \pi_\alpha \oplus x_\beta \oplus \pi_\beta = x_\gamma \oplus \pi_\gamma .$$

Therefore, the correctness of BitGC-fast can follow that of the BitGC-small scheme.

**Security proof.** The proof of BitGC-fast follows the same hybrid-based strategy as in BitGC-small, except for some modifications tailored to the construction of BitGC-fast. Similarly, we first change the garbling process without relying on secret key $\Delta$. Then, $\llbracket \Delta \rrbracket$ and $\llbracket s_i \rrbracket$ for $i \in [\ell]$ are replaced with encryptions of zero. The changes are computationally indistinguishable from the real-world execution, under the assumption that the SWHEwDD scheme satisfies the IND-CPA and KDM security and PRF is a secure PRF. Next, each bit $r_\gamma^{i,j}$ is replaced by a uniform bit, justified by the pseudorandomness of the PRG. Finally, the 5-bit

communication values for each AND gate are shown to be uniformly random, since they are masked with independent random bits. Note that permutation bits are uniform and independent of the adversary's view in the final hybrid. Through the above proof idea, we are able to obtain a proof for privacy. Using the same approach as in the proof of BitGC-small, we can easily obtain the proofs for both obliviousness and authenticity.

**Theorem 2.** *Garbling scheme* BitGC-fast *shown in Figures 6 and 7 satisfies privacy, obliviousness and authenticity, provided that the SWHEwDD scheme satisfies the IND-CPA and KDM security properties,* PRG *is a secure PRG,* PRF *is a secure PRF, and* H *is a random oracle.*

The proof of the above theorem is postponed to Appendix B.2.

### 5.2 BitGC-tiny with Free-XOR, 1 Bit per AND Gate and More Computational Cost

We describe a garbling scheme called BitGC-tiny, which supports free-XOR and communication of 1 bit per AND gate. While BitGC-tiny achieves the lowest communication cost among our three garbling schemes, it requires the usage of a leveled HE scheme and brings about much more computational cost. A leveled homomorphic encryption with distributed decryption (LHEwDD) scheme follows the same syntax and security properties as the SWHEwDD scheme (Section 3.3), except that the maximum multiplication depth $L$ of the leveled HE scheme can be any positive integer.

The detailed construction of the BitGC-tiny scheme is shown in Figure 8. The BitGC-tiny scheme shares the En and De algorithms with the BitGC-small scheme (Section 4.1). Thus, these two algorithms are omitted for BitGC-tiny. In the following, we discuss the main differences between BitGC-tiny and BitGC-small.

- In BitGC-tiny, the leveled HE scheme requires $\lceil \log(\ell_{\mathsf{XOR}}+1) \rceil$ more levels of homomorphic multiplications than BitGC-small, where $\ell_{\mathsf{XOR}}$ is the length of the longest consecutive sequence of XOR gates in the circuit.
- Homomorphic PRG computation needs to generate the HE ciphertexts on $|\mathcal{I}| + |\mathcal{W}_{\mathsf{AND}}|$ (instead of $|\mathcal{I}| + |\mathcal{W}|$) pseudorandom bits.
- For each XOR gate $(\alpha, \beta, \gamma, \mathsf{XOR})$, $\pi_\gamma$ is directly defined as $\pi_\alpha \oplus \pi_\beta$ rather than an output bit of $\mathsf{PRG}(\boldsymbol{s})$. Following the approach in BitGC-fast, the bit $d_\gamma = \pi_\alpha \oplus \pi_\beta \oplus \pi_\gamma = 0$ need not be sent. Nevertheless, both the Gb and Ev algorithms still need to compute the ciphertext $[\![\pi_\gamma]\!]$, as it remains necessary for garbling subsequent AND gates. To minimize the multiplicative depth, we recursively expand $\pi_\gamma$ into its base constituent masks whenever an input itself is an output of an XOR gate. The final $[\![\pi_\gamma]\!]$ then is computed as the homomorphic XOR sum of all flattened masks.

Based on the correctness analysis of BitGC-small and BitGC-fast, the correctness of the BitGC-tiny scheme can be naturally derived. By combining the proof of BitGC-small with that of BitGC-fast, it is easy to obtain the proof of BitGC-tiny, when replacing the IND-CPA and KDM security of SWHEwDD with that of LHEwDD. We state the security of the BitGC-tiny scheme in the following theorem.

**Theorem 3.** *Garbling scheme* BitGC-tiny *shown in Figure 8 satisfies privacy, obliviousness and authenticity, provided that the LHEwDD scheme satisfies the IND-CPA and KDM security properties,* PRG *is a secure PRG,* PRF *is a secure PRF, and* H *is a random oracle.*

The proof of the above theorem can be found in Appendix B.3.

BitGC-tiny

Let $\mathsf{LHEwDD} = (\mathsf{Setup}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ be a leveled HE with distributed decryption scheme. Let $\mathsf{PRG} : \{0,1\}^{\ell} \to \{0,1\}^{N}$ be a secure $t$-depth PRG where $\ell$ is the seed size and $N = |\mathcal{I}| + |\mathcal{W}_{\mathsf{AND}}|$, $\mathsf{PRF} : \{0,1\}^{\lambda} \times \{0,1\}^{\lambda} \to \mathcal{R}_p$ be a secure PRF, and $\mathsf{H} : \mathcal{R}_p \to \{0,1\}^{\lambda}$ be a random oracle.

$\mathsf{Gb}(1^{\lambda}, f)$:

1. Let $\ell_{\mathsf{XOR}}$ be the length of the longest consecutive sequence of XOR gates in the circuit $f$. Generate a set of setup parameters $\mathsf{params} \leftarrow \mathsf{Setup}(1^{\lambda}, 1^{L})$, where $L = \lceil \log(\ell_{\mathsf{XOR}} + 1) \rceil + t + 2$ is the maximum multiplication depth for the leveled HE scheme.
2. Produce a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params})$ where $\mathsf{LSB}(\Delta) = 1$, and generate $[\![\Delta]\!] := \mathsf{Enc}(\Delta, \Delta)$. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^{\lambda}$.
3. Sample a PRG seed $\boldsymbol{s} \xleftarrow{\$} \{0,1\}^{\ell}$ and generate a ciphertext $[\![s_i]\!] := \mathsf{Enc}(\Delta, s_i)$ for each bit $s_i$ with $i \in [\ell]$. Then, compute $\{\pi_\gamma\}_{\gamma \in \mathcal{I} \cup \mathcal{W}_{\mathsf{AND}}} := \mathsf{PRG}(\boldsymbol{s})$, and homomorphically compute ciphertexts $\{[\![\pi_\gamma]\!]\}_{\gamma \in \mathcal{I} \cup \mathcal{W}_{\mathsf{AND}}} := \mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$.
4. For each input wire $i \in \mathcal{I}$, sample a label $W_i^0 \xleftarrow{\$} \mathcal{R}_p$ and compute $W_i^1 := W_i^0 + (-1)^{\pi_i} \cdot \Delta \in \mathcal{R}_p$.
5. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, compute a bit $\hat{z}^{i,j} := (\pi_\alpha \oplus i) \wedge (\pi_\beta \oplus j) \oplus \pi_\gamma$ and homomorphically compute a ciphertext $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!]$, $[\![\pi_\beta]\!]$, $[\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$ with two levels of homomorphic multiplications.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute a pair of labels $W_\gamma^{\pi_\gamma} := \mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) - \hat{z}^{0,0} \cdot \Delta + \mathsf{PRF}_K(\gamma)$ and $W_\gamma^{\pi_\gamma \oplus 1} := W_\gamma^{\pi_\gamma} + \Delta$, along with a bit $d_\gamma := \mathsf{LSB}(W_\gamma^0) \oplus \pi_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute $\pi_\gamma := \pi_\alpha \oplus \pi_\beta$, and express $\pi_\gamma$ as a subset XOR sum of $\{\pi_\gamma\}_{\gamma \in \mathcal{I} \cup \mathcal{W}_{\mathsf{AND}}}$. Homomorphically evaluate $[\![\pi_\gamma]\!]$ by computing the XOR sum of the corresponding ciphertexts. Set $\tau_1 := [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (b) Compute a pair of labels $W_\gamma^{\pi_\gamma} := \mathsf{Eval}_{0,0}(W_\alpha^{\pi_\alpha}, W_\beta^{\pi_\beta}, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$ and $W_\gamma^{\pi_\gamma \oplus 1} := W_\gamma^{\pi_\gamma} + \Delta$.
6. Output a garbled circuit $F := (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{[\![s_i]\!]\}_{i \in [\lambda]}, \{d_i\}_{i \in \mathcal{W}_{\mathsf{AND}}})$, an encoding information $e := (\{(\pi_i, W_i^0)\}_{i \in \mathcal{I}}, \Delta)$ and a decoding information $d := \{(\mathsf{H}(W_i^0), \mathsf{H}(W_i^1))\}_{i \in \mathcal{O}}$.

$\mathsf{Ev}(F, X)$:

1. Homomorphically compute $\{[\![\pi_\gamma]\!]\}_{\gamma \in \mathcal{I} \cup \mathcal{W}_{\mathsf{AND}}} := \mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$.
2. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, perform the following steps:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!]$, $[\![\pi_\beta]\!]$, $[\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, and $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Compute a masked bit $\hat{x}_\gamma := \mathsf{LSB}(W_\gamma) \oplus d_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Homomorphically evaluate $[\![\pi_\gamma]\!]$ in the same way as the Gb algorithm does.
     (b) Set $\tau_1 := [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Compute a masked bit $\hat{x}_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta$.
3. Output $Y := \{W_i\}_{i \in \mathcal{O}}$.

Figure 8: Garbling scheme BitGC-tiny with 1 bit per AND gate and free-XOR, where the differences between BitGC-tiny and BitGC-small are marked in blue.

# 6 Implementation Details

We implement the proposed garbling schemes BitGC-fast and BitGC-small[1] in `C++` on top of `Microsoft SEAL 4.1` [SEA23]. In this section, we provide the details of the implementation, including the instantiation of the SWHEwDD scheme and the low-depth PRG, homomorphic evaluation of the PRG with SIMD optimization, and concrete parameters.

**SWHEwDD instantiation.** We adopt the BGV framework [BGV12] under the RLWE assumption as our basic SWHE scheme. We instantiate distributed decryption on $k = 1$ following the blueprint proposed by Chillotti et al. [COS+22], and instantiate that on $k = 2$ by providing a BGV-native instantiation that replaces the GSW-style ciphertexts in the original BitGC scheme [LWYY25b]. We further adopt the CP-RLWE-based approach proposed in recent work [ILL25b] for the $k = 2$ case, for better efficiency. As a result, our SWHEwDD instantiation relies on RLWE for the base SWHE and the $k = 1$ distributed decryption case, and on CP-RLWE for the $k = 2$ case, substantially improving concrete efficiency over the existing instantiation [LWYY25b]. We defer the details of the resulting SWHEwDD construction to Appendix A. We note that the SWHEwDD scheme can also be instantiated under the BFV framework [Bra12, FV12] in a similar way.

**Low-depth PRG instantiation.** We instantiate our low-depth PRG using Goldreich-style random local PRGs [Gol11]. Let $d$ be the locality of the PRG, $\ell$ be the seed size, $\ell^s$ for some $s > 1$ be the polynomial stretch, and $P : \{0,1\}^d \to \{0,1\}$ be the predicate. The $i$-th bit of the PRG output $y_i$ is associated with a randomly sampled public index set $I_i \in [\ell]^d$, and is evaluated by the predicate $P$ on the input bits indexed by $I_i$. We choose the predicate $P$ as $\mathsf{XOR}_{k_1}$-$\mathsf{Majority}_{k_2}$ proposed by Applebaum and Lovett [AL16], and use the parameters as in [ABBS25] with $k_1 = 4$ and $k_2 = 7$ for a high stretch. We set parameters $\ell = 2^{16}$, $s = 2$, and stretch $\ell^s = 2^{32}$, which can resist the best known attacks [OW14, Üna23, YGJL22]. The predicate $P$ is defined as

$$\begin{aligned} P(\{x_i\}_{i\in[11]}) &= \mathsf{Majority}\left(\{x_i\}_{i\in[7]}\right) \oplus (x_7 \oplus x_8 \oplus x_9 \oplus x_{10}) \\ &= \left(\bigoplus_{0\le i<j<k<l<7} (x_i \wedge x_j \wedge x_k \wedge x_l)\right) \oplus (x_7 \oplus x_8 \oplus x_9 \oplus x_{10})\,. \end{aligned}$$

Since the plaintext space in our SWHEwDD instantiation is a polynomial ring $\mathcal{R}_p$, we must arithmetize the Boolean predicate $P$ to enable homomorphic evaluation. We achieve this by mapping logical operations to arithmetic ones: $a \wedge b \mapsto a \cdot b$ and $a \oplus b \mapsto a + b - 2 \cdot a \cdot b$. We further simplify the expression by reducing all higher-degree exponents ($x^k \equiv x$ for $x \in \{0,1\}$). The resulting arithmetic representation of $P$ consists of additive terms of up to 11 degrees, which can be homomorphically evaluated with a multiplicative depth of 4.

**SIMD-enabled PRG evaluation.** The ring structure of the plaintext space $\mathcal{R}_p$ naturally supports SIMD batching. Let $N$ be the ring degree and $m$ be the number of packed slots ($m$ is typically $N$ under full packing). An input of $m \cdot \ell$ seed bits, divided into $m$ blocks of $\ell$ bits each, is packed into $\ell$ polynomials and then encrypted, where the $i$-th polynomial encodes the $i$-th bit of all $m$ blocks across its slots. We then homomorphically evaluate the arithmetic expression of the PRG over the ciphertexts, and yield each output ciphertext encrypting a packed polynomial of $m$ bits.

For Gb and Ev algorithms, we need single-bit ciphertexts rather than packed outputs. Therefore, we further apply an unpacking pipeline to extract $m$ single-bit ciphertexts from a packed ciphertext. It can be achieved in two steps. First, we perform slot-to-coefficient conversion using the BSGS method [HS15, HS18],

[1] Source code is available at https://github.com/wtd2/more-efficient-bitgc/. Compared to the conference version [ZLY+26], this implementation adopts a tightened modulus-chain configuration that improves concrete efficiency without changing protocol functionality; the resulting parameters appear in the SWHEwDD parameters paragraph below.

which requires $O(2\sqrt{N})$ rotations and one multiplicative layer. Then, we run coefficient expansion to obtain individual bit ciphertexts [CDKS21], which requires $O(N)$ homomorphic operations and no additional multiplicative depth. For better concrete efficiency, before the coefficient expansion, we switch the ciphertexts to a smaller modulus/dimension context via ring-dimension reduction [CDKS21].

**SWHEwDD parameters.** Our implementation uses BGV in residue number system (RNS) form. Let $(N_{\mathsf{PRG}}, Q_{\mathsf{PRG}})$ denote the large modulus/dimension context used for packed PRG evaluation, and $(N_{\mathsf{Gate}}, Q_{\mathsf{Gate}})$ the small context after the dimension reduction used for gate evaluation, with $N_{\mathsf{PRG}} > N_{\mathsf{Gate}}$. Ciphertext moduli are represented as RNS chains of 60-bit data limbs. For distributed decryption, we use two reduced moduli $q$ and $r$ for $k = 1$ and $k = 2$ cases respectively (details in Appendix A).

We choose parameters to target a per-gate failure probability below $2^{-40}$ and a 128-bit RLWE security (estimated following [ACC+19]). Concretely, we set $\log p \approx 60$ as the plaintext modulus, $\log q \approx 120$ and $\log r \approx 240$ as the moduli for $k = 1$ and $k = 2$ distributed decryption. For BitGC-fast, we use moduli $\log Q_{\mathsf{PRG}} \approx 600, \log Q_{\mathsf{Gate}} \approx 180$ with 10 and 3 data limbs, and $N_{\mathsf{PRG}} = 2^{15}, N_{\mathsf{Gate}} = 2^{13}$ correspondingly. For BitGC-small, we use moduli $\log Q_{\mathsf{PRG}} \approx 780, \log Q_{\mathsf{Gate}} \approx 300$ with 13 and 5 data limbs, and $N_{\mathsf{PRG}} = 2^{15}, N_{\mathsf{Gate}} = 2^{14}$ correspondingly.

# 7 Performance Evaluation

In this section, we perform a series of benchmark experiments to study the concrete performance of the BitGC scheme and the proposed variants BitGC-fast, BitGC-small, and BitGC-tiny.

**Experimental setup.** Our experiments are conducted on an AWS EC2 instance of type `m8azn.24xlarge`, with up to 384 GiB of memory. We conduct all experiments with *single-threaded* execution for simplicity. To evaluate the concrete performance of our schemes, we benchmark the end-to-end garbling and evaluation pipeline. We structure the pipeline into two phases:

- **Preprocessing**: this stage requires only an upper bound on the circuit size, and consists primarily of homomorphic PRG evaluation over the encrypted seed. For our improved SIMD-enabled schemes, two steps are involved: **(1) HE-PRG** which homomorphically evaluates the PRG over encrypted packed polynomials; **(2) Extraction** which extracts individual ciphertexts from the packed ciphertexts. For the original BitGC, only the HE-PRG step is involved. For the BitGC-fast, we also account for the cost of evaluating $[\![r \cdot \Delta]\!]$ from $[\![r]\!]$ and $[\![\Delta]\!]$ for each PRG expanded pseudorandom bit $r$ into the extraction step.
- **Gate-dependent**: this execution requires the circuit topology. During this phase, the garbler generates the garbled table, while the evaluator takes as input the garbled table and evaluates the output label of each gate in a topological order. It consists of **(3) Assembly** which assembles three ciphertexts as the input of the function $\mathsf{Eval}_{i,j}$. **(4) DD** which applies the distributed-decryption family DD on the assembled ciphertexts and the input labels to obtain the output label. For BitGC-tiny, each XOR gate additionally incurs a homomorphic ciphertext multiplication to derive the output wire's permutation-bit ciphertext, which we account for in the Assembly step.

**Experimental results.** We evaluate the performance of our schemes BitGC-fast and BitGC-small by benchmarking an AES-128 circuit with 6,400 AND gates and 28,176 XOR gates. The circuit also contains 2,087 NOT gates, whose cost is almost negligible. We utilize a single thread for execution, and note that the preprocessing phase can be further accelerated by parallelizing the execution, as each PRG output bit is associated with an independent index set. Since the computation costs of the garbling and evaluation are nearly identical, we only report the results of the garbling pipeline for simplicity.

In addition, we provide a lower-bound performance estimation for the original BitGC scheme and our BitGC-tiny scheme. For the original BitGC, we estimate the performance by assuming a non-SIMD homomorphic PRG evaluation and assigning the cost of an AND gate to all gates; the per-gate gate-dependent

| | Gate | Comm. | Preprocessing | | Gate-dependent | | Total time |
|---|---|---|---|---|---|---|---|
| | | | HE-PRG | Extraction | Assembly | DD | |
| BitGC-fast | AND | 5 | 0.11 | 15.11 | 0.07 | 0.95 | 16.24 |
| | XOR | 0 | 0 | 0 | 0 | 0.43 | 0.43 |
| BitGC-small | AND | 1 | 0.04 | 3.52 | 28.57 | 2.07 | 34.20 |
| | XOR | 1 | | | 0 | 0.91 | 4.47 |
| BitGC-tiny | AND | 1 | $\geq 0.2$ | $\geq 34$ | $\geq 277$ | $\geq 20$ | $\geq 331$ |
| | XOR | 0 | 0 | 0 | $\geq 35$ | $\geq 9$ | $\geq 44$ |
| BitGC [LWYY25b] | AND | 1 | $\geq 1418$ | 0 | $\geq 28.57$ | $\geq 2.07$ | $\geq 1449$ |
| | XOR | 1 | | | | | |

Table 2: **Performance comparison between BitGC and our proposed variants.** The communication cost (bits), amortized running time (ms) in total, and amortized running time for each pipeline step (ms) are reported for each AND and XOR gate. Costs for BitGC and BitGC-tiny are lower-bound estimation. Costs are amortized over large circuits and exclude reusable data. The communication cost of transmitting an encrypted seed is negligibly small compared to other components of a garbled circuit and thus is omitted.

cost is taken to inherit the implementation-level optimizations of our BitGC-small variant, so the resulting estimate is a lower bound on what a faithful implementation of the original BitGC construction would achieve. For the performance estimation of BitGC-tiny, we assume that the additional multiplication depth for homomorphic evaluation is bounded by 5 (supporting circuits with up to 31 consecutive XOR operations). Correspondingly, we set $\log Q_{\mathsf{PRG}} = 1200, \log Q_{\mathsf{Gate}} = 660, N_{\mathsf{PRG}} = 2^{16}, N_{\mathsf{Gate}} = 2^{15}$ for the parameters underlying the BitGC-tiny scheme. For each XOR gate, we account for the cost of homomorphic computing the permutation bit into the "assembly" step.

We note that our instantiated low-depth PRG has an expansion factor of $\ell^{s-1}$, and we enable SIMD for homomorphic PRG evaluation. Therefore, the amortized communication cost for transmitting encrypted seeds is $\log Q_{\mathsf{PRG}}/\ell^{s-1} < 0.02$ bits per homomorphically expanded pseudorandom bit. We need 1 pseudorandom bit per gate for BitGC-small and BitGC, 4 pseudorandom bits per AND gate for BitGC-fast, and 1 pseudorandom bit per AND gate for BitGC-tiny. We therefore omit the amortized communication cost for transmitting encrypted seeds.

The one-time setup communication of SWHEwDD consists of the public parameters and key-related material (the ciphertext of the secret key $\Delta$, preprocessed ciphertexts for distributed decryption, and all evaluation keys). Measured on our prototype with 128-bit-secure parameters, the setup payload is about 1.6 GiB for BitGC-fast and about 2.7 GiB for BitGC-small. This cost is independent of the circuit size and can be amortized to negligibly small for sufficiently large circuits. Compared to the best-known "Yao-style" garbled circuit [RR21], which sends $1.5\lambda + 5$ bits per AND gate and no bits per XOR gate, BitGC-fast achieves lower total communication once more than approximately $7 \times 10^7$ AND gates (about 11,000 AES-128 circuits) are evaluated.

The amortized communication and time costs per gate, along with the time costs for each pipeline step, are presented in Table 2.

**Analysis.** As shown in the table, BitGC scheme incurs a substantial overhead from its non-SIMD homomorphic PRG evaluation. In contrast, with SIMD optimization, our BitGC-small scheme significantly reduces the time cost by over $400\times$ end-to-end on AES-128, with the preprocessing phase alone seeing about $400\times$. Furthermore, by applying specific optimizations for XOR gates rather than treating them generically,

BitGC-small scheme further cuts more than $30\times$ the cost during the gate-dependent phase for XOR gates.

We also note that BitGC-fast scheme provides a powerful trade-off between communication and concrete computational cost. It significantly reduces the per-gate time cost for AND and XOR gates by approximately $2\times$ and $10\times$, respectively, compared to BitGC-small scheme, and substantially further compared to the BitGC-tiny scheme. We acknowledge that it requires more communication per AND gate, but BitGC-fast scheme benefits from the free-XOR property that eliminates communication for XOR gates. This can lead to a comparable total communication cost to BitGC-small scheme, especially in circuits with a high ratio of XOR gates to AND gates like AES-128.

## Acknowledgements

The work of Yu Yu is supported by the National Natural Science Foundation of China (Grant Nos. 62125204 and 92270201) and Innovation Program for Quantum Science and Technology (No. 2021ZD0302901 / 2021ZD0302902).

## References

[ABBS25] Amit Agarwal, Carsten Baum, Lennart Braun, and Peter Scholl. Low-bandwidth mixed arithmetic in VOLE-based ZK from low-degree PRGs. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 396–426. Springer, Cham, May 2025.

[ACC+19] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. Homomorphic encryption standard. Cryptology ePrint Archive, Report 2019/939, 2019.

[Agr17] Shweta Agrawal. Stronger security for reusable garbled circuits, general definitions and attacks. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 3–35. Springer, Cham, August 2017.

[AL16] Benny Applebaum and Shachar Lovett. Algebraic attacks against random local functions and their countermeasures. In Daniel Wichs and Yishay Mansour, editors, *48th ACM STOC*, pages 1087–1100. ACM Press, June 2016.

[Ale03] Michael Alekhnovich. More on average case vs approximation complexity. In *44th FOCS*, pages 298–307. IEEE Computer Society Press, October 2003.

[App12] Benny Applebaum. Pseudorandom generators with long stretch and low locality from random local one-way functions. In Howard J. Karloff and Toniann Pitassi, editors, *44th ACM STOC*, pages 805–816. ACM Press, May 2012.

[AR16] Benny Applebaum and Pavel Raykov. Fast pseudorandom functions based on expander graphs. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part I*, volume 9985 of *LNCS*, pages 27–56. Springer, Berlin, Heidelberg, October / November 2016.

[BGG+14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth

Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014.

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012.

[BHKR13] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. Efficient garbling from a fixed-key blockcipher. In *2013 IEEE Symposium on Security and Privacy*, pages 478–492. IEEE Computer Society Press, May 2013.

[BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796. ACM Press, October 2012.

[BKS19] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 3–33. Springer, Cham, May 2019.

[BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513. ACM Press, May 1990.

[BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 719–737. Springer, Berlin, Heidelberg, April 2012.

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012.

[BRS03] John Black, Phillip Rogaway, and Thomas Shrimpton. Encryption-scheme security in the presence of key-dependent messages. In Kaisa Nyberg and Howard M. Heys, editors, *SAC 2002*, volume 2595 of *LNCS*, pages 62–75. Springer, Berlin, Heidelberg, August 2003.

[CDKS21] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. In Kazue Sako and Nils Ole Tippenhauer, editors, *ACNS 2021, Part I*, volume 12726 of *LNCS*, pages 460–479. Springer, Cham, June 2021.

[COS+22] Ilaria Chillotti, Emmanuela Orsini, Peter Scholl, Nigel P. Smart, and Barry van Leeuwen. Scooby: Improved multi-party homomorphic secret sharing based on FHE. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 540–563. Springer, Cham, September 2022.

[DG17] Nico Döttling and Sanjam Garg. Identity-based encryption from the Diffie-Hellman assumption. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 537–569. Springer, Cham, August 2017.

[FNO15] Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 191–219. Springer, Berlin, Heidelberg, April 2015.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.

[GGH+13] Craig Gentry, Sergey Gorbunov, Shai Halevi, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. How to compress (reusable) garbled circuits. Cryptology ePrint Archive, Report 2013/687, 2013.

[GKP+13] Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. Reusable garbled circuits and succinct functional encryption. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 555–564. ACM Press, June 2013.

[GKW+20] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 793–822. Springer, Cham, August 2020.

[GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020.

[GLNP15] Shay Gueron, Yehuda Lindell, Ariel Nof, and Benny Pinkas. Fast garbling of circuits under standard assumptions. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 567–578. ACM Press, October 2015.

[Gol11] Oded Goldreich. *Candidate One-Way Functions Based on Expander Graphs*, pages 76–87. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.

[GWYY25] Chun Guo, Xiao Wang, Kang Yang, and Yu Yu. On tweakable correlation robust hashing against key leakages. *DCC*, 93(8):3375–3412, 2025.

[HK20] David Heath and Vladimir Kolesnikov. Stacked garbling for disjunctive zero-knowledge proofs. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 569–598. Springer, Cham, May 2020.

[HKE12] Yan Huang, Jonathan Katz, and David Evans. Quid-Pro-Quo-tocols: Strengthening semi-honest protocols with dual execution. In *2012 IEEE Symposium on Security and Privacy*, pages 272–284. IEEE Computer Society Press, May 2012.

[HPS98] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In Joe P. Buhler, editor, *Algorithmic Number Theory*, pages 267–288, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.

[HS15] Shai Halevi and Victor Shoup. Bootstrapping for HElib. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 641–670. Springer, Berlin, Heidelberg, April 2015.

[HS18] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HElib. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 93–120. Springer, Cham, August 2018.

[IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003.

[ILL25a] Yuval Ishai, Hanjun Li, and Huijia Lin. Succinct homomorphic MACs from groups and applications. In *66th FOCS*, pages 2505–2545. IEEE Computer Society Press, December 2025.

[ILL25b] Yuval Ishai, Hanjun Li, and Huijia Lin. A unified framework for succinct garbling from homomorphic secret sharing. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part IV*, volume 16003 of *LNCS*, pages 390–425. Springer, Cham, August 2025.

[JKO13] Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 955–966. ACM Press, November 2013.

[JRR25] Jake Januzelli, Mike Rosulek, and Lawrence Roy. Lower bounds for garbled circuits from shannon-type information inequalities. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part IV*, volume 16003 of *LNCS*, pages 589–618. Springer, Cham, August 2025.

[KMR14] Vladimir Kolesnikov, Payman Mohassel, and Mike Rosulek. FleXOR: Flexible garbling for XOR gates that beats free-XOR. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 440–457. Springer, Berlin, Heidelberg, August 2014.

[KP17] Yashvanth Kondi and Arpita Patra. Privacy-free garbled circuits for formulas: Size zero and information-theoretic. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 188–222. Springer, Cham, August 2017.

[KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, Berlin, Heidelberg, July 2008.

[LP09] Yehuda Lindell and Benny Pinkas. A proof of security of Yao's protocol for two-party computation. *Journal of Cryptology*, 22(2):161–188, April 2009.

[LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010.

[LWYY25a] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. Authenticated BitGC for actively secure rate-one 2PC. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part IV*, volume 16003 of *LNCS*, pages 652–687. Springer, Cham, August 2025.

[LWYY25b] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. BitGC: Garbled circuits with 1 bit per gate. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 437–466. Springer, Cham, May 2025.

[MORS25] Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Silent circuit relinearisation: Sublinear-size (boolean and arithmetic) garbled circuits from DCR. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part IV*, volume 16003 of *LNCS*, pages 426–458. Springer, Cham, August 2025.

[NPS99] Moni Naor, Benny Pinkas, and Reuban Sumner. Privacy preserving auctions and mechanism design. In *Proceedings of the 1st ACM Conference on Electronic Commerce*, pages 129–139, New York, NY, USA, 1999. Association for Computing Machinery.

[OW14] Ryan ODonnell and David Witmer. Goldreich's PRG: evidence for near-optimal polynomial stretch. In *2014 IEEE 29th Conference on Computational Complexity (CCC)*, pages 1–12, 2014.

[PS21] Alice Pellet-Mary and Damien Stehlé. On the hardness of the NTRU problem. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 3–35. Springer, Cham, December 2021.

[PSSW09] Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267. Springer, Berlin, Heidelberg, December 2009.

[RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 2021. Springer, Cham.

[SEA23] Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL, January 2023. Microsoft Research, Redmond, WA.

[Üna23] Akin Ünal. Worst-case subexponential attacks on PRGs of constant degree or constant locality. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 25–54. Springer, Cham, April 2023.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.

[YGJL22] Jing Yang, Qian Guo, Thomas Johansson, and Michael Lentmaier. Revisiting the concrete security of goldreich's pseudorandom generator. *IEEE Transactions on Information Theory*, 68(2):1329–1354, 2022.

[ZLY+26] Wenhao Zhang, Hanlin Liu, Kang Yang, Wen-jie Lu, Yu Yu, Xiao Wang, and Chenkai Weng. BitGC made (more) efficient. In *EUROCRYPT 2026*. Springer, Cham, May 2026.

[ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250. Springer, Berlin, Heidelberg, April 2015.

# A SWHEwDD Instantiation

## A.1 Hardness Assumptions

**Definition 7** (RLWE assumption)**.** *Let $n = n(\lambda) \in \mathbb{N}$ be a ring degree, $p = p(\lambda), q = q(\lambda) \in \mathbb{N}$ be a plaintext and a ciphertext modulus respectively, $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ be the corresponding cyclotomic ring, and let $\mathcal{D}_\Delta, \mathcal{D}_e$ be efficiently sampleable distributions over $\mathcal{R}$ indexed by $\lambda$. The RLWE assumption (with parameters $(n, p, q, \mathcal{D}_\Delta, \mathcal{D}_e)$) states that for every polynomial $m = m(\lambda)$, the distribution*

$$\left\{ (\boldsymbol{a}, \boldsymbol{b}) \;\middle|\; \Delta \leftarrow \mathcal{D}_\Delta, \boldsymbol{e} \leftarrow \mathcal{D}_e^m, \boldsymbol{a} \xleftarrow{\$} \mathcal{R}_q^m, \boldsymbol{b} = \boldsymbol{a} \cdot \Delta + p \cdot \boldsymbol{e} \right\}$$

*is computationally indistinguishable from $\{(\boldsymbol{a}, \boldsymbol{u}) \mid \boldsymbol{a}, \boldsymbol{u} \xleftarrow{\$} \mathcal{R}_q^m\}$.*

**Definition 8** (Circular Power RLWE assumption, CP-RLWE [ILL25b])**.** *Let $n, p, \mathcal{R}, \mathcal{D}_\Delta, \mathcal{D}_e$ be as in the RLWE definition above, and let $r = r(\lambda) \in \mathbb{N}$ be a ciphertext modulus. The circular power RLWE (CP-RLWE) assumption (with parameters $(n, p, r, \mathcal{D}_\Delta, \mathcal{D}_e)$) states that for every polynomial $m = m(\lambda)$, the distribution*

$$\left\{ (\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}) \;\middle|\; \begin{array}{l} \Delta \leftarrow \mathcal{D}_\Delta, \boldsymbol{e}_1, \boldsymbol{e}_2 \leftarrow \mathcal{D}_e^m, \boldsymbol{a} \xleftarrow{\$} \mathcal{R}_r^m, \\ \boldsymbol{b} = \boldsymbol{a} \cdot \Delta + p \cdot \boldsymbol{e}_1, \boldsymbol{c} = \boldsymbol{a} \cdot \Delta^2 + \Delta + p \cdot \boldsymbol{e}_2 \end{array} \right\}$$

*is computationally indistinguishable from $\{(\boldsymbol{a}, \boldsymbol{u}, \boldsymbol{v}) \mid \boldsymbol{a}, \boldsymbol{u}, \boldsymbol{v} \xleftarrow{\$} \mathcal{R}_r^m\}$.*

## A.2 Overview of RLWE-based BGV Scheme

We recall the level-structured BGV scheme of Brakerski, Gentry, and Vaikuntanathan [BGV12], retaining only the parameter conventions and noise-budget invariant on which our construction relies; we treat the homomorphic operations as black boxes and refer to [BGV12, HS15, HS18] for their constructions.

**Parameters and key material.** The scheme realizes the SWHE interface $\mathsf{BGV} = (\mathsf{Setup}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ of Definition 5, parameterized by a security parameter $\lambda$ and a target multiplicative depth $L$. For our distributed-decryption analysis we will only need the ring degree $n$, the plaintext modulus $p$, the level-0 ciphertext modulus $q$, the secret-key distribution $\mathcal{D}_\Delta$ (taken to be uniform ternary, so $\|\Delta\|_\infty \le 1$), and the noise distribution $\mathcal{D}_e$ (a centered discrete Gaussian on $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ with standard deviation $\sigma_e$).

**Ciphertext form.** A level-0 BGV ciphertext is a pair $\tau = (\boldsymbol{a}, \boldsymbol{b}) \in \mathcal{R}_q^2$ that encrypts a message $m \in \mathcal{R}_p$ via $\boldsymbol{b} - \boldsymbol{a} \cdot \Delta \equiv p \cdot \boldsymbol{e} + m \pmod q$ for some noise $\boldsymbol{e} \in \mathcal{R}$, with decryption $\mathsf{Dec}(\Delta, \tau) = ((\boldsymbol{b} - \boldsymbol{a} \cdot \Delta) \bmod q) \bmod p$. For any depth-$L$ circuit, $\mathsf{HEval}$ produces a level-0 output whose noise satisfies $\|\boldsymbol{e}\|_\infty \le E_{\text{out}}$ with overwhelming probability over the random coins of $\mathsf{Setup}$, $\mathsf{Gen}$, and $\mathsf{HEval}$, for some bound $E_{\text{out}} = E_{\text{out}}(L, n, p, \sigma_e)$ polynomial in the depth and parameters and independent of the plaintexts. We choose parameters to satisfy the distributed-decryption condition $n \cdot (E_{\text{out}} + 1) \cdot p/q \le \mathsf{negl}(\lambda)$ of Lemma 1. We assume throughout this section that the input to DD is such a level-0 ciphertext.

## A.3 Extended BGV Scheme for SWHEwDD Instantiation

We instantiate an SWHEwDD scheme $\mathsf{BGVwDD} = (\mathsf{Setup}_{\mathsf{DD}}, \mathsf{Gen}_{\mathsf{DD}}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval}, \mathsf{DD})$ that satisfies the distributed decryption property of Definition 6 on both $k = 1$ and $k = 2$, under the RLWE and CP-RLWE assumptions, respectively. The encryption, decryption, and homomorphic-evaluation algorithms $(\mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ are inherited from the BGV scheme; the new ingredients are the augmented setup and key-generation algorithms $(\mathsf{Setup}_{\mathsf{DD}}, \mathsf{Gen}_{\mathsf{DD}})$, which extend $(\mathsf{Setup}, \mathsf{Gen})$ with the parameters and preprocessed ciphertexts used by distributed decryption, and the distributed decryption algorithms $\mathsf{DD}_x$ (for $k = 1$)

and $\mathsf{DD}_{x,y}$ (for $k = 2$). The construction of $\mathsf{DD}_x$ follows the linear-DD blueprint of Boyle, Kohl, and Scholl [BKS19], instantiated for BGV in [COS+22]; the $k = 2$ algorithm $\mathsf{DD}_{x,y}$ adapts the lattice construction of Ishai, Li, and Lin [ILL25b] using CP-RLWE-shaped preprocessed ciphertexts. We assume throughout this section that the input ciphertext to DD is a level-0 ciphertext $\tau = (\boldsymbol{a}, \boldsymbol{b}) \in \mathcal{R}_q^2$ satisfying the noise invariant with some bound $E$.

**Shared randomness.** The garbler and the evaluator have access to a common source of synced randomness that supplies, on each DD invocation, an identical pair of fresh uniform masking values to both parties: $R$ over $\mathcal{R}_q$ or $\mathcal{R}_r$ (depending on context), and (for $k = 2$) an additional $\bar{R}$ over $\mathcal{R}_p$. Distinct invocations use independent masking values, even when applied to the same input ciphertext. The analysis below treats each such value as a truly uniform sample from the appropriate ring.

**Parameters and key-generation.** Let $r = r(\lambda) \in \mathbb{N}$ be the $k = 2$ distributed-decryption modulus, chosen so that the parameter conditions of Lemma 2 hold. The augmented setup and key-generation algorithms are then defined as follows.

- $\mathsf{Setup}_{\mathsf{DD}}(1^\lambda, 1^L)$ runs $\mathsf{params} \leftarrow \mathsf{Setup}(1^\lambda, 1^L)$ and outputs the augmented parameters $\mathsf{params}_{\mathsf{DD}} := (\mathsf{params}, r)$. (We treat the shared randomness used by DD as set up alongside $\mathsf{params}_{\mathsf{DD}}$, as described above.)
- $\mathsf{Gen}_{\mathsf{DD}}(\mathsf{params}_{\mathsf{DD}})$ runs $(\mathsf{keypar}, \Delta) \leftarrow \mathsf{Gen}(\mathsf{params})$, samples $\boldsymbol{a} \xleftarrow{\$} \mathcal{R}_r$ and $\boldsymbol{e}_1, \boldsymbol{e}_2 \leftarrow \mathcal{D}_e$, and computes the CP-RLWE-shaped triple $(\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3) := (\boldsymbol{a},\ \boldsymbol{a} \cdot \Delta + p \cdot \boldsymbol{e}_1,\ \boldsymbol{a} \cdot \Delta^2 + \Delta + p \cdot \boldsymbol{e}_2) \bmod r$. It outputs the augmented key-related parameters $\mathsf{keypar}_{\mathsf{DD}} := (\mathsf{keypar}, (\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3))$ together with the secret key $\Delta$.

**Distributed decryption on $k = 1$.** Given the augmented parameters $(\mathsf{params}_{\mathsf{DD}}, \mathsf{keypar}_{\mathsf{DD}})$ output by $\mathsf{Setup}_{\mathsf{DD}}$ and $\mathsf{Gen}_{\mathsf{DD}}$, we define the distributed decryption algorithm $\mathsf{DD}_x : \mathcal{R}_p \times \mathcal{C} \to \mathcal{R}_p$, parameterized by $x \in \{0, 1\}$, by

$$\mathsf{DD}_x(X, \tau) := \big(x \cdot \boldsymbol{b} - X \cdot \boldsymbol{a} + R\big) \bmod q \bmod p,$$

where $\tau = (\boldsymbol{a}, \boldsymbol{b}) \in \mathcal{R}_q^2$ is a level-0 ciphertext and $R \in \mathcal{R}_q$ is the shared masking value for this invocation. The algorithm $\mathsf{DD}_x$ takes $(\mathsf{params}_{\mathsf{DD}}, \mathsf{keypar}_{\mathsf{DD}})$ as implicit input.

**Lemma 1.** *Let $\tau = (\boldsymbol{a}, \boldsymbol{b}) \in \mathcal{R}_q^2$ be a level-0 ciphertext such that $\boldsymbol{b} - \boldsymbol{a} \cdot \Delta \equiv p \cdot \boldsymbol{e} + m \pmod q$ for some $m = \mathsf{Dec}(\Delta, \tau) \in \mathcal{R}_p$ and noise $\boldsymbol{e} \in \mathcal{R}$ with $\|\boldsymbol{e}\|_\infty \le E$. Suppose the parameters satisfy $n \cdot (\|\Delta\|_\infty + 1)/p \le \mathsf{negl}(\lambda)$ and $n \cdot (E + 1) \cdot p/q \le \mathsf{negl}(\lambda)$. Then for every $x \in \{0, 1\}$, over the random choice of $X \xleftarrow{\$} \mathcal{R}_p$ and $R \xleftarrow{\$} \mathcal{R}_q$,*

$$\Pr\left[\mathsf{DD}_x(X + x \cdot \Delta, \tau) = \mathsf{DD}_0(X, \tau) + x \cdot m\right] \ge 1 - \mathsf{negl}(\lambda).$$

*Proof.* The case $x = 0$ is immediate, since both sides of the equation reduce to $\mathsf{DD}_0(X, \tau)$. Assume henceforth $x = 1$. By the lifting lemma [BKS19] applied modulo $p$, with probability at least $1 - n \cdot (\|\Delta\|_\infty + 1)/p$ over the choice of $X \xleftarrow{\$} \mathcal{R}_p$ we have $(X + \Delta) \bmod p = X + \Delta$ over $\mathcal{R}$, so that

$$\mathsf{DD}_1(X + \Delta, \tau) - \mathsf{DD}_0(X, \tau) = \big((\boldsymbol{b} - (X + \Delta) \cdot \boldsymbol{a} + R) \bmod q - (R - X \cdot \boldsymbol{a}) \bmod q\big) \bmod p.$$

Since $R$ is uniform over $\mathcal{R}_q$, the value $(R - X \cdot \boldsymbol{a}) \bmod q$ is also uniform over $\mathcal{R}_q$. Substituting $\boldsymbol{b} = \boldsymbol{a} \cdot \Delta + p \cdot \boldsymbol{e} + m \pmod q$ and applying the lifting lemma a second time, this time modulo $q$, with probability at least $1 - n \cdot (\|\boldsymbol{e}\|_\infty + 1) \cdot p/q$ we obtain

$$\begin{aligned}
&\big((\boldsymbol{b} - (X + \Delta) \cdot \boldsymbol{a} + R) \bmod q - (R - X \cdot \boldsymbol{a}) \bmod q\big) \bmod p \\
&= \big((R - X \cdot \boldsymbol{a}) \bmod q + p \cdot \boldsymbol{e} + m - (R - X \cdot \boldsymbol{a}) \bmod q\big) \bmod p \\
&= (p \cdot \boldsymbol{e} + m) \bmod p \ = \ m.
\end{aligned}$$

Union-bounding over the two lifting-lemma applications, and using $\|\boldsymbol{e}\|_\infty \le E$ together with the parameter conditions in the lemma statement, we obtain

$$\begin{aligned}\Pr\left[\mathsf{DD}_1(X+\Delta,\tau) = \mathsf{DD}_0(X,\tau)+m\right] &\ge 1 - n\cdot\big((\|\Delta\|_\infty+1)/p + (E+1)\cdot p/q\big)\\ &\ge 1-\mathsf{negl}(\lambda),\end{aligned}$$

which concludes the proof. □

**Distributed decryption on $k=2$.** Given the augmented key-related parameters $\mathsf{keypar}_{\mathsf{DD}} = (\mathsf{keypar}, (\boldsymbol{c}_1,\boldsymbol{c}_2,\boldsymbol{c}_3))$ produced by $\mathsf{Gen}_{\mathsf{DD}}$, we define the auxiliary function $F_{x,y}:\mathcal{R}_p^2\to\mathcal{R}_p$ by

$$F_{x,y}(X,Y) := \big(X\cdot Y\cdot\boldsymbol{c}_1 - (x\cdot Y + y\cdot X)\cdot\boldsymbol{c}_2 + x\cdot y\cdot\boldsymbol{c}_3 + R\big) \bmod r \bmod p,$$

where $R\in\mathcal{R}_r$ is the shared masking value for this $F_{x,y}$ invocation. The $k=2$ distributed decryption algorithm $\mathsf{DD}_{x,y}:\mathcal{R}_p^2\times\mathcal{C}\to\mathcal{R}_p$ is then defined by composing $F_{x,y}$ with the $k=1$ algorithm $\mathsf{DD}_{xy}$:

$$\mathsf{DD}_{x,y}(X,Y,\tau) := \mathsf{DD}_{xy}\big(F_{x,y}(X,Y)+\bar{R},\ \tau\big),$$

where $\bar{R}\in\mathcal{R}_p$ is an additional shared masking value that re-randomizes the input to $\mathsf{DD}_{xy}$ over $\mathcal{R}_p$ so that the uniform-first-argument premise of Lemma 1 is satisfied.

**Lemma 2.** *Let $(\boldsymbol{c}_1,\boldsymbol{c}_2,\boldsymbol{c}_3)$ be the CP-RLWE-shaped triple produced by $\mathsf{Gen}_{\mathsf{DD}}$ from noise samples $\boldsymbol{e}_1,\boldsymbol{e}_2\in\mathcal{R}$, and let $E_1,E_2$ satisfy $\|\boldsymbol{e}_1\|_\infty\le E_1$ and $\|\boldsymbol{e}_2\|_\infty\le E_2$. Suppose the parameters satisfy $2n\cdot(\|\Delta\|_\infty+1)/p\le\mathsf{negl}(\lambda)$ and $n\cdot(E_2+4n\cdot p\cdot E_1+1)\cdot p/r\le\mathsf{negl}(\lambda)$. Then for every $x,y\in\{0,1\}$, over the random choice of $X,Y\xleftarrow{\$}\mathcal{R}_p$ and $R\xleftarrow{\$}\mathcal{R}_r$,*

$$\Pr\left[F_{x,y}(X+x\cdot\Delta, Y+y\cdot\Delta) = F_{0,0}(X,Y) + xy\cdot\Delta\right]\ge 1-\mathsf{negl}(\lambda).$$

*Proof.* Let $\boldsymbol{e}(x,y) = xy\cdot\boldsymbol{e}_2 - (2xy\cdot\Delta + x\cdot Y + y\cdot X)\cdot\boldsymbol{e}_1$ over $\mathcal{R}$ (before reduction modulo $r$). Using the standard bound $\|f\cdot g\|_\infty\le n\cdot\|f\|_\infty\cdot\|g\|_\infty$ over $\mathcal{R}$, together with $\|\Delta\|_\infty\le 1$, $\|X\|_\infty,\|Y\|_\infty\le p$ for $X,Y\in\mathcal{R}_p$, and $x,y\in\{0,1\}$, we have

$$\begin{aligned}\|\boldsymbol{e}(x,y)\|_\infty &\le \|\boldsymbol{e}_2\|_\infty + n\cdot(2\|\Delta\|_\infty + \|X\|_\infty + \|Y\|_\infty)\cdot\|\boldsymbol{e}_1\|_\infty\\ &\le E_2 + n\cdot(2+2p)\cdot E_1 \ \le\ E_2 + 4n\cdot p\cdot E_1.\end{aligned}$$

Based on the lifting lemma [BKS19], with at least $1-2n\cdot(\|\Delta\|_\infty+1)/p$ probability, we have both $(X+x\cdot\Delta)\bmod p = X + x\cdot\Delta$ and $(Y+y\cdot\Delta)\bmod p = Y+y\cdot\Delta$ over $\mathcal{R}$; therefore,

$$\begin{aligned}&F_{x,y}(X+x\cdot\Delta, Y+y\cdot\Delta) - F_{0,0}(X,Y)\\ &= \big((X\cdot Y\cdot\boldsymbol{a} + R + p\cdot\boldsymbol{e}(x,y) + xy\cdot\Delta) \bmod r - (X\cdot Y\cdot\boldsymbol{a}+R)\bmod r\big)\bmod p.\end{aligned}$$

Since $R\xleftarrow{\$}\mathcal{R}_r$ is independent of the rest, $(X\cdot Y\cdot\boldsymbol{a}+R)\bmod r$ is uniform over $\mathcal{R}_r$ and independent of $\boldsymbol{e}(x,y)$. Applying the lifting lemma a second time, with probability at least $1-n\cdot(\|\boldsymbol{e}(x,y)\|_\infty+1)\cdot p/r$,

$$\begin{aligned}&((X\cdot Y\cdot\boldsymbol{a} + R + p\cdot\boldsymbol{e}(x,y) + xy\cdot\Delta)\bmod r - (X\cdot Y\cdot\boldsymbol{a}+R)\bmod r)\bmod p\\ &= ((X\cdot Y\cdot\boldsymbol{a}+R)\bmod r + p\cdot\boldsymbol{e}(x,y) + xy\cdot\Delta - (X\cdot Y\cdot\boldsymbol{a}+R)\bmod r)\bmod p\\ &= (p\cdot\boldsymbol{e}(x,y) + xy\cdot\Delta)\bmod p = xy\cdot\Delta.\end{aligned}$$

Substituting the bound on $\|\boldsymbol{e}(x,y)\|_\infty$ derived above and union-bounding over the two lifting-lemma applications, we obtain

$$\begin{aligned}&\Pr\left[F_{x,y}(X + x\cdot\Delta, Y + y\cdot\Delta) = F_{0,0}(X,Y) + xy\cdot\Delta\right]\\ &\geq 1 - 2n\cdot(\|\Delta\|_\infty + 1)/p - n\cdot(E_2 + 4n\cdot p\cdot E_1 + 1)\cdot p/r\\ &\geq 1 - \mathsf{negl}(\lambda),\end{aligned}$$

where the final inequality follows from the parameter conditions in the lemma statement. This concludes the proof. □

Combining Lemma 1 with Lemma 2 yields the $k = 2$ correctness statement for the composed algorithm $\mathsf{DD}_{x,y}$.

**Corollary 1.** *Let $\tau = (\boldsymbol{a}, \boldsymbol{b}) \in \mathcal{R}_q^2$ be a level-0 ciphertext such that $\boldsymbol{b} - \boldsymbol{a}\cdot\Delta \equiv p\cdot\boldsymbol{e} + m \pmod q$, where $m = \mathsf{Dec}(\Delta,\tau)$ and $\|\boldsymbol{e}\|_\infty \leq E$, and let $E_1, E_2$ bound the noise samples used by $\mathsf{Gen}_{\mathsf{DD}}$ as in Lemma 2. Suppose the parameters satisfy the conditions of Lemmas 1 and 2 simultaneously. Then for every $x, y \in \{0,1\}$, over the random choice of $X, Y \xleftarrow{\$} \mathcal{R}_p$, $R \xleftarrow{\$} \mathcal{R}_r$, and $\bar{R} \xleftarrow{\$} \mathcal{R}_p$,*

$$\Pr\left[\mathsf{DD}_{x,y}(X + x\cdot\Delta, Y + y\cdot\Delta, \tau) = \mathsf{DD}_{0,0}(X, Y, \tau) + xy\cdot m\right] \geq 1 - \mathsf{negl}(\lambda).$$

The proof is by direct substitution: Lemma 2 gives $F_{x,y}(X + x\Delta, Y + y\Delta) = F_{0,0}(X,Y) + xy\Delta$ except with negligible probability; the additive shift $\bar{R}$ is identical on both sides of the equation; and the outer $\mathsf{DD}_{xy}$ call (whose first argument is uniform over $\mathcal{R}_p$ thanks to $\bar{R}$) preserves the correlation by Lemma 1.

**Theorem 4.** *Suppose the BGV scheme $\mathsf{BGV} = (\mathsf{Setup}, \mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{HEval})$ satisfies IND-CPA and KDM security with respect to the secret-key distribution $\mathcal{D}_\Delta$ and noise distribution $\mathcal{D}_e$, the RLWE assumption holds with parameters $(n, p, q, \mathcal{D}_\Delta, \mathcal{D}_e)$, the CP-RLWE assumption holds with parameters $(n, p, r, \mathcal{D}_\Delta, \mathcal{D}_e)$, and the parameters satisfy the conditions of Lemmas 1 and 2. Then, the scheme $\mathsf{BGVwDD}$ constructed above, with $\mathsf{DD} = \{\mathsf{DD}_x, \mathsf{DD}_{x,y}\}_{x,y\in\{0,1\}}$, is an SWHEwDD scheme satisfying Definition 6 on $k = 1$ and $k = 2$, and inherits the IND-CPA and KDM security of $\mathsf{BGV}$.*

*Sketch.* We argue correctness and security separately.
*Correctness.* The $k = 1$ case follows from Lemma 1 and the $k = 2$ case from Corollary 1, treating the shared masking randomness as truly uniform as discussed above.
*Security.* The encryption and decryption algorithms of BGVwDD are inherited verbatim from BGV, so the only additional material in the augmented key-related parameters is the CP-RLWE-shaped triple $(\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3)$. Under the CP-RLWE assumption, this triple is computationally indistinguishable from a uniform tuple over $\mathcal{R}_r^3$, after which $\mathsf{keypar}_{\mathsf{DD}}$ is independent of $\Delta$ beyond what keypar already reveals. Any IND-CPA or KDM adversary against BGVwDD thus reduces to one against BGV with the same advantage. □

# B Security Proofs

## B.1 Security Proof of BitGC-small

**Theorem 5** (Theorem 1, restated)**.** *Garbling scheme* BitGC-small *shown in Figures 4 and 5 satisfies privacy, obliviousness and authenticity, provided that the SWHEwDD scheme satisfies the IND-CPA and KDM security properties,* PRG *is a secure PRG,* PRF *is a secure PRF, and* H *is a random oracle.*

$\mathcal{S}(1^\lambda, f, f(x))$: $\mathcal{S}$ answers the queries of random oracle $\mathsf{H}$ by returning consistent random values, and simulates the $\mathsf{Gb}$ and $\mathsf{En}$ algorithms as follows:

1. Generate a set of setup parameters $\mathsf{params} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^L)$, where $L = t + 2$ is the maximum multiplication depth for $\mathsf{SWHE}$.
2. Produce a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \xleftarrow{\$} \mathsf{Gen}(\mathsf{params})$ where $\mathsf{LSB}(\Delta) = 1$, and generate a ciphertext $[\![\Delta]\!] := \mathsf{Enc}(\Delta, 0)$.
3. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^\lambda$. For each $i \in [\ell]$, run $\sigma_i := \mathsf{Enc}(\Delta, 0)$. Homomorphically compute a tuple of ciphertexts $([\![\pi_0]\!], \ldots, [\![\pi_{N-1}]\!]) := \mathsf{PRG}(\sigma_0, \ldots, \sigma_{\lambda-1})$.
4. For each input wire $i \in \mathcal{I}$, sample a label $W_i \xleftarrow{\$} \mathcal{R}_p$ and a masked bit $\hat{x}_i \xleftarrow{\$} \{0,1\}$.
5. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!]$, $[\![\pi_\beta]\!]$, $[\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Sample $d_\gamma \xleftarrow{\$} \{0,1\}$ and set a masked bit $\hat{x}_\gamma := \mathsf{LSB}(W_\gamma) \oplus d_\gamma$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Sample $d_\gamma \xleftarrow{\$} \{0,1\}$ and set a masked bit $\hat{x}_\gamma := d_\gamma \oplus \hat{x}_\alpha \oplus \hat{x}_\beta$.
     (b) Compute $\tau_1 := (1 - 2 \cdot d_\gamma) \cdot [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
6. For each $i \in \mathcal{I}$, set $X_i := (\hat{x}_i, W_i)$. For each $i \in \mathcal{O}$, sample $V_i \xleftarrow{\$} \{0,1\}^\lambda$, and set
$$\mathsf{H}_i := \begin{cases} (\mathsf{H}(W_i),\ V_i) & \text{if } x_i = 0 \\ (V_i,\ \mathsf{H}(W_i)) & \text{if } x_i = 1 \end{cases}.$$
7. Output a garbled circuit $F = (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{\sigma_i\}_{i \in [\ell]}, \{d_i\}_{i \in \mathcal{W}})$, a set of input labels $X = \{X_i\}_{i \in \mathcal{I}}$ and a decoding information $d = \{\mathsf{H}_i\}_{i \in \mathcal{O}}$.

Figure 9: The construction of simulator $\mathcal{S}$ for privacy of BitGC-small.

*Proof.* The proof of obliviousness follows directly from the proof of privacy. The only difference is that we do not need to simulate the decoding information $d$ for the proof of obliviousness. The rest of the obliviousness proof proceeds in exactly the same way as the proof of privacy, showing that the garbled circuit and input labels constructed by a simulator are computationally indistinguishable from those in the real-world execution.

We first show the proof for privacy. Specifically, we construct a probabilistic polynomial time (PPT) simulator shown in Figure 9. We prove the ideal-world execution is computationally indistinguishable from the real-world execution via a series of hybrids.

**Hybrid 0.** This is the real-world execution and denoted by $\mathcal{G}_0$. This hybrid generates a transcript $(F, X, d)$ as shown in Figure 4.

**Hybrid 1.** This hybrid (denoted by $\mathcal{G}_1$) is the same as $\mathcal{G}_0$, except that the transcript $(F, X, d)$ is generated using the $\mathsf{Gb}$ and $\mathsf{En}$ algorithms shown in Figure 10. In particular, the steps 4, 5, and 6 change the calculation of labels to eliminate the dependence on the secret key $\Delta$. This hybrid still knows the input $x$.

The differences between $\mathcal{G}_0$ and $\mathcal{G}_1$ are listed as follows:

1. For each input wire $i \in \mathcal{I}$, the garbled input $X_i = (\hat{x}_i, W_i)$. Here, $W_i$ is sampled uniformly from $\mathcal{R}_p$ in

$\mathcal{G}_1(1^\lambda, f, x)$: This hybrid answers the queries of random oracle $\mathsf{H}$ by returning consistent random values, and simulates the $\mathsf{Gb}$ and $\mathsf{En}$ algorithms as follows:

1. Generate a set of setup parameters $\mathsf{params} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^L)$, where $L = t + 2$ is the maximum multiplication depth for SWHEwDD.
2. Produce a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \xleftarrow{\$} \mathsf{Gen}(\mathsf{params})$ where $\mathsf{LSB}(\Delta) = 1$, and generate a ciphertext $[\![\Delta]\!] := \mathsf{Enc}(\Delta, \Delta)$.
3. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^\lambda$. Sample a PRG seed $\boldsymbol{s} \xleftarrow{\$} \{0,1\}^\ell$ and generate a ciphertext $[\![s_i]\!] := \mathsf{Enc}(\Delta, s_i)$ for each bit $s_i$ with $i \in [\ell]$. Then, compute $(\pi_0, \ldots, \pi_{N-1}) := \mathsf{PRG}(\boldsymbol{s})$, and homomorphically compute a tuple of ciphertexts $([\![\pi_0]\!], \ldots, [\![\pi_{N-1}]\!]) := \mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$.
4. For each input wire $i \in \mathcal{I}$, sample a label $W_i \xleftarrow{\$} \mathcal{R}_p$ and set a masked bit $\hat{x}_i := x_i \oplus \pi_i$.
5. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, compute the real value $x_\gamma \in \{0,1\}$ with $(f, x)$, and do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!]$, $[\![\pi_\beta]\!]$, $[\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Compute a masked bit $\hat{x}_\gamma := x_\gamma \oplus \pi_\gamma$ and a bit $d_\gamma := \hat{x}_\gamma \oplus \mathsf{LSB}(W_\gamma)$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute a masked bit $\hat{x}_\gamma := x_\gamma \oplus \pi_\gamma$ and a bit $d_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta \oplus \hat{x}_\gamma$.
     (b) Compute $\tau_1 := (1 - 2 \cdot d_\gamma) \cdot [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
6. For each $i \in \mathcal{I}$, set $X_i := (\hat{x}_i, W_i)$. For each $i \in \mathcal{O}$, sample $V_i \xleftarrow{\$} \{0,1\}^\lambda$, and set
$$\mathsf{H}_i := \begin{cases} (\mathsf{H}(W_i),\ V_i) & \text{if } x_i = 0 \\ (V_i,\ \mathsf{H}(W_i)) & \text{if } x_i = 1 \end{cases}.$$
7. Output a garbled circuit $F := (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{[\![s_i]\!]\}_{i \in [\ell]}, \{d_i\}_{i \in \mathcal{W}})$, a set of input labels $X = \{X_i\}_{i \in \mathcal{I}}$ and a decoding information $d := \{\mathsf{H}_i\}_{i \in \mathcal{O}}$.

Figure 10: The hybrid $\mathcal{G}_1$ used in the proof of BitGC-small, where the differences are marked in blue compared to the real world $\mathcal{G}_0$.

$\mathcal{G}_1$, while $W_i = W_i^0 + (-1)^{\pi_i} \cdot x_i \cdot \Delta$ for a uniform element $W_i^0 \in \mathcal{R}_p$ in $\mathcal{G}_0$.

2. For each gate $(\alpha, \beta, \gamma, T)$, the label $W_\gamma$ associated with the real value $x_\gamma$ is computed in $\mathcal{G}_1$ as
$$W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)\ ,$$
while in $\mathcal{G}_0$, $W_\gamma$ is derived from the base label $W_\gamma^{\pi_\gamma}$.
3. For each $i \in \mathcal{O}$, $\mathsf{H}_i$ included in the decoding information is computed as
$$\mathsf{H}_i := \begin{cases} (\mathsf{H}(W_i),\ V_i) & \text{if } x_i = 0 \\ (V_i,\ \mathsf{H}(W_i)) & \text{if } x_i = 1 \end{cases}$$
where $V_i \xleftarrow{\$} \{0,1\}^\lambda$ in $\mathcal{G}_1$, while in $\mathcal{G}_0$, we set $\mathsf{H}_i := (\mathsf{H}(W_i^0),\ \mathsf{H}(W_i^1))$.

$\mathcal{G}_2(1^\lambda, f, x)$ // $\boxed{\mathcal{G}_3(1^\lambda, f, x)}$ : This hybrid simulates the random oracle $\mathsf{H}$ honestly, and simulates the $\mathsf{Gb}$ and $\mathsf{En}$ algorithms as follows:

1. Generate a set of setup parameters $\mathsf{params} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^L)$, where $L = t + 2$ is the maximum multiplication depth for SWHEwDD.
2. Produce a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \xleftarrow{\$} \mathsf{Gen}(\mathsf{params})$ where $\mathsf{LSB}(\Delta) = 1$, and generate a ciphertext $[\![\Delta]\!] := \mathsf{Enc}(\Delta, 0)$.
3. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^\lambda$. Sample a PRG seed $\boldsymbol{s} \xleftarrow{\$} \{0,1\}^\ell$. For each $i \in [\ell]$, run $\sigma_i := \mathsf{Enc}(\Delta, 0)$.
   Compute $(\pi_0, \ldots, \pi_{N-1}) := \mathsf{PRG}(\boldsymbol{s})$. // $\boxed{\text{Sample } \pi_i \xleftarrow{\$} \{0,1\} \text{ for each } i \in [N].}$
   Homomorphically compute a tuple of ciphertexts $([\![\pi_0]\!], \ldots, [\![\pi_{N-1}]\!]) := \mathsf{PRG}\,(\sigma_0, \ldots, \sigma_{\ell-1})$.
4. For each input wire $i \in \mathcal{I}$, sample a label $W_i \xleftarrow{\$} \mathcal{R}_p$ and set a masked bit $\hat{x}_\gamma := x_\gamma \oplus \pi_\gamma$.
5. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, compute the real value $x_\gamma \in \{0,1\}$ with $(f, x)$, do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!]$ from $[\![\pi_\alpha]\!]$, $[\![\pi_\beta]\!]$, $[\![\pi_\gamma]\!]$ and $[\![\Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Compute a masked bit $\hat{x}_\gamma := x_\gamma \oplus \pi_\gamma$, and a bit $d_\gamma := \hat{x}_\gamma \oplus \mathsf{LSB}(W_\gamma)$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute a masked bit $\hat{x}_\gamma := x_\gamma \oplus \pi_\gamma$ and a bit $d_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta \oplus \hat{x}_\gamma$.
     (b) Compute $\tau_1 := (1 - 2 \cdot d_\gamma) \cdot [\![\Delta]\!]$, $\tau_2 := \tau_1$ and $\tau_3 := -\tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
6. For each $i \in \mathcal{I}$, set $X_i := (\hat{x}_i, W_i)$. For each $i \in \mathcal{O}$, sample $V_i \xleftarrow{\$} \{0,1\}^\lambda$, and set
$$\mathsf{H}_i := \begin{cases} (\mathsf{H}(W_i),\ V_i) & \text{if } x_i = 0 \\ (V_i,\ \mathsf{H}(W_i)) & \text{if } x_i = 1 \end{cases}.$$
7. Output a garbled circuit $F := (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{\sigma_i\}_{i \in [\ell]}, \{d_i\}_{i \in \mathcal{W}})$, a set of input labels $X = \{X_i\}_{i \in \mathcal{I}}$ and a decoding information $d := \{\mathsf{H}_i\}_{i \in \mathcal{O}}$.

Figure 11: The hybrids $\mathcal{G}_2$ and $\mathcal{G}_3$ used in the proof, where the differences are marked in blue compared to $\mathcal{G}_1$. The difference between $\mathcal{G}_2$ and $\mathcal{G}_3$ is marked by a rectangular box.

In $\mathcal{G}_0$, for each $i \in \mathcal{I}$, $W_i = W_i^0 + (-1)^{\pi_i} \cdot x_i \cdot \Delta$ is uniform in $\mathcal{R}_p$, since $W_i^0$ is sampled uniformly from $\mathcal{R}_p$. In $\mathcal{G}_1$, for each $i \in \mathcal{I}$, $W_i \in \mathcal{R}_p$ is sampled at random. Thus, for each $i \in \mathcal{I}$, $W_i$ in $\mathcal{G}_1$ has the identical distribution as that in $\mathcal{G}_0$. Based on the correctness analysis (Section 4.2), we can see that the second change does not bring about a noticeable difference between $\mathcal{G}_0$ and $\mathcal{G}_1$, under the assumption that the outputs of $\mathsf{PRF}$ are pseudorandom.

The third change replaces $\mathsf{H}(W_i^{1-x_i})$ with a uniform string $V_i$. We prove that any PPT adversary cannot make a query $W_i^{1-x_i} = W_i + (-1)^{\pi_i} \cdot (1 - 2x_i) \cdot \Delta$ to random oracle $\mathsf{H}$. Otherwise, we can construct an algorithm $\mathcal{B}$ to break the IND-CPA and KDM security of the SWHEwDD scheme. In particular, $\mathcal{B}$ simulates the random oracle $\mathsf{H}$, and records the queries to $\mathsf{H}$. Algorithm $\mathcal{B}$ behaves just like in $\mathcal{G}_1$, except that all ciphertexts are generated by querying its encryption oracle. Then, $\mathcal{B}$ is able to efficiently extract a secret key $\Delta$ from the queries of $\mathsf{H}$ at the end, and use $\Delta$ to break the IND-CPA and KDM security. Therefore, no adversary makes a query $W_i^{1-x_i}$ to $\mathsf{H}$, meaning that $\mathsf{H}(W_i^{1-x_i})$ is computationally indistinguishable from a uniform string $V_i$ in the random oracle model.

In conclusion, $\mathcal{G}_1$ is computationally indistinguishable from $\mathcal{G}_0$.

**Hybrid 2.** This hybrid (denoted by $\mathcal{G}_2$) is the same as $\mathcal{G}_1$, except that the ciphertext $[\![\Delta]\!]$ and $\{[\![s_i]\!]\}_{i\in[\ell]}$ are replaced by fresh encryptions on zero, i.e., $\mathsf{Enc}(\Delta, 0)$ (see Figure 11).

In $\mathcal{G}_1$, $[\![\Delta]\!]$ encrypts the secret key $\Delta$, and each $[\![s_i]\!]$ encrypts a bit $s_i$ under the same key $\Delta$. In $\mathcal{G}_2$, all these ciphertexts are replaced with fresh encryptions of 0. It is straightforward to bound the difference between $\mathcal{G}_1$ and $\mathcal{G}_2$ by using a reduction to the IND-CPA and KDM security of the SWHEwDD scheme.

**Hybrid 3.** This hybrid (denoted by $\mathcal{G}_3$) is the same as $\mathcal{G}_2$, except that each bit $\pi_i$ is sampled uniformly at random (see Figure 11).

The only change is how each bit $\pi_i$ with $i \in [N]$ is generated: in $\mathcal{G}_2$, they are the outputs of $\mathsf{PRG}(\boldsymbol{s})$, while in $\mathcal{G}_3$, they are uniform bits. Since the seed $\boldsymbol{s}$ is now uniform and independent of the ciphertexts $\sigma_0, \ldots, \sigma_{\ell-1}$, $\mathcal{G}_3$ is computationally indistinguishable from $\mathcal{G}_2$ based on the pseudorandomness of $\mathsf{PRG}$ outputs.

**Hybrid 4.** This hybrid (denoted by $\mathcal{G}_4$) is the same as $\mathcal{G}_3$, except for the following differences:
1. For each $i \in \mathcal{I}$, the masked bit $\hat{x}_i$ is sampled uniformly.
2. For each $\gamma \in \mathcal{W}$, the bit $d_\gamma$ is picked uniformly at random.

Specifically, $\mathcal{G}_4$ generates the transcript $(F, X, d)$ as shown in Figure 9. That is, $\mathcal{G}_4$ is the same as the simulator. Now, this is the ideal world.

In $\mathcal{G}_3$, $\pi_i$ is uniformly random in $\{0,1\}$, and thus $\hat{x}_i = x_i \oplus \pi_i$ is a uniform bit. In $\mathcal{G}_4$, $\hat{x}_i \in \{0,1\}$ is sampled uniformly. The first change does not bring about any difference. The second change is related to how each bit $d_\gamma$ is generated. In $\mathcal{G}_3$, each $d_\gamma \in \{0,1\}$ is computed as $d_\gamma := \hat{x}_\gamma \oplus \mathsf{LSB}(W_\gamma) = x_\gamma \oplus \pi_\gamma \oplus \mathsf{LSB}(W_\gamma)$ for an AND gate and $d_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta \oplus \hat{x}_\gamma = \hat{x}_\alpha \oplus \hat{x}_\beta \oplus x_\gamma \oplus \pi_\gamma$ for an XOR gate, where $\pi_\gamma \in \{0,1\}$ is uniform. In $\mathcal{G}_4$, each $d_\gamma \in \{0,1\}$ is sampled uniformly. Therefore, the distribution w.r.t. $d_\gamma$ is identical between $\mathcal{G}_3$ and $\mathcal{G}_4$. That is, $\mathcal{G}_4$ is perfectly indistinguishable from $\mathcal{G}_3$.

In the following, we prove that the BitGC-small scheme satisfies authenticity. Specifically, we use the simulator from the proof of privacy shown in Figure 9 to generate the adversary's view. From the proof of privacy, we have that it is computationally indistinguishable from the real view. According to the above analysis, we also obtain that a PPT adversary never makes a query $W_i^{1-x_i}$ to random oracle $\mathsf{H}$, under the assumption that the SWHEwDD scheme satisfies the IND-CPA and KDM security. Therefore, for each $i \in \mathcal{O}$, $\mathcal{A}$ has to guess a uniform string $V_i$ with probability $1/2^\lambda$. In conclusion, the probability that $\mathcal{A}$ forges a garbled output $\widehat{Y}$, such that $\mathsf{De}(d, \widehat{Y}) \neq \perp$ and $\widehat{Y} \neq \mathsf{Ev}(F, X)$ is negligible in $\lambda$, which completes the proof. □

## B.2 Security Proof of BitGC-fast

**Theorem 6** (Theorem 2, restated)**.** *Garbling scheme* BitGC-fast *shown in Figures 6 and 7 satisfies privacy, obliviousness and authenticity, provided that the SWHEwDD scheme satisfies the IND-CPA and KDM security properties,* PRG *is a secure PRG,* PRF *is a secure PRF, and* H *is a random oracle.*

*Sketch.* We show that the garbling scheme BitGC-fast achieves privacy, obliviousness, and authenticity. Since the overall proof follows the structure of Theorem 1, we provide a sketch. In particular, we present a new simulator in Figure 12, where all modifications compared to the simulator of Theorem 1 are explicitly highlighted in blue.

The proof proceeds through a sequence of hybrids. First, we replace each input label $W_i$ with a uniform element from $\mathcal{R}_p$. The replacement is statistically indistinguishable from the real world, because $W_i$ is already uniformly distributed in the real world. Next, we replace the uniform string $V_i$ with $\mathsf{H}(W_i^{1-x_i})$, where $W_i^{1-x_i}$ denotes the hidden label of output wire $i$. If an adversary could query $W_i^{1-x_i}$, we could extract the secret key $\Delta$ and break the IND-CPA and KDM security of the SWHEwDD scheme, which is impossible. Thus, $\mathsf{H}(W_i^{1-x_i})$

$\mathcal{S}(1^\lambda, f, f(x))$: $\mathcal{S}$ answers the queries of random oracle $\mathsf{H}$ by returning consistent random values, and simulates the $\mathsf{Gb}$ and $\mathsf{En}$ algorithms as follows:

1. Generate a set of setup parameters $\mathsf{params} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^L)$, where $L = t + 1$ is the maximum multiplication depth for SWHEwDD.
2. Produce a set of key-dependent parameters $(\mathsf{keypar}, \Delta) \xleftarrow{\$} \mathsf{Gen}(\mathsf{params})$ where $\mathsf{LSB}(\Delta) = 1$, and generate a ciphertext $[\![\Delta]\!] := \mathsf{Enc}(\Delta, 0)$.
3. Sample a PRF key $K \xleftarrow{\$} \{0,1\}^\lambda$. For each $i \in [\ell]$, run $\sigma_i := \mathsf{Enc}(\Delta, 0)$. For each $\gamma \in \mathcal{W}_{\mathsf{AND}}$ and $i, j \in \{0,1\}$, sample $r_\gamma^{i,j} \leftarrow \{0,1\}$ uniformly at random, and then compute a ciphertext $[\![r_\gamma^{i,j}]\!]$ by homomorphically evaluating $\mathsf{PRG}([\![s_0]\!], \ldots, [\![s_{\ell-1}]\!])$.
4. For each $\gamma \in \mathcal{W}_{\mathsf{AND}}$ and $i, j \in \{0,1\}$, homomorphically compute a ciphertext $[\![r_\gamma^{i,j} \cdot \Delta]\!]$ from $[\![r_\gamma^{i,j}]\!]$ and $[\![\Delta]\!]$.
5. For each input wire $i \in \mathcal{I}$, sample a label $W_i \xleftarrow{\$} \mathcal{R}_p$ and a masked bit $\hat{x}_i \xleftarrow{\$} \{0,1\}$.
6. For each gate $(\alpha, \beta, \gamma, T)$ in topological order, do the following:
   - Case 1 ($T = \mathsf{AND}$):
     (a) For each $i, j \in \{0,1\}$, sample $d_\gamma^{i,j} \xleftarrow{\$} \{0,1\}$ and homomorphically compute $[\![\hat{z}^{i,j} \cdot \Delta]\!] := d_\gamma^{i,j} \cdot [\![\Delta]\!] + (1 - 2 \cdot d_\gamma^{i,j}) \cdot [\![r_\gamma^{i,j} \cdot \Delta]\!]$.
     (b) Compute $\tau_1 := [\![\hat{z}^{1,0} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_2 := [\![\hat{z}^{0,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!]$, $\tau_3 := [\![\hat{z}^{1,1} \cdot \Delta]\!] - [\![\hat{z}^{0,0} \cdot \Delta]\!] - \tau_1 - \tau_2$.
     (c) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
     (d) Sample $d_\gamma \xleftarrow{\$} \{0,1\}$ and set a masked bit $\hat{x}_\gamma := d_\gamma \oplus \mathsf{LSB}(W_\gamma)$.
   - Case 2 ($T = \mathsf{XOR}$):
     (a) Compute $\hat{x}_\gamma := \hat{x}_\alpha \oplus \hat{x}_\beta$, along with three ciphertexts $\tau_1 := [\![\Delta]\!]$, $\tau_2 := \tau_1$, and $\tau_3 := -\tau_1 - \tau_2$.
     (b) Compute $W_\gamma := \mathsf{Eval}_{\hat{x}_\alpha, \hat{x}_\beta}(W_\alpha, W_\beta, \tau_1, \tau_2, \tau_3) + \mathsf{PRF}_K(\gamma)$.
7. For each $i \in \mathcal{I}$, set $X_i := (\hat{x}_i, W_i)$. For each $i \in \mathcal{O}$, sample $V_i \xleftarrow{\$} \{0,1\}^\lambda$, and set
$$\mathsf{H}_i := \begin{cases} (\mathsf{H}(W_i),\ V_i) & \text{if } x_i = 0 \\ (V_i,\ \mathsf{H}(W_i)) & \text{if } x_i = 1 \end{cases}.$$
8. Output a garbled circuit $F := (\mathsf{params}, \mathsf{keypar}, [\![\Delta]\!], K, \{\sigma_i\}_{i \in [\ell]}, \{(d_\gamma, d_\gamma^{0,0}, d_\gamma^{0,1}, d_\gamma^{1,0}, d_\gamma^{1,1})\}_{\gamma \in \mathcal{W}_{\mathsf{AND}}})$, a set of input labels $X = \{X_i\}_{i \in \mathcal{I}}$ and a decoding information $d := \{\mathsf{H}_i\}_{i \in \mathcal{O}}$.

Figure 12: The construction of the simulator for the proof of garbling scheme BitGC-fast. The differences between the simulator for BitGC-fast and that for BitGC-small are marked in blue.

is indistinguishable from $V_i$ in the random-oracle model. Next, we replace all ciphertexts, including $[\![\Delta]\!]$ and each $[\![s_i]\!]$, with fresh encryptions of 0. This modification yields computationally indistinguishable hybrids, guaranteed by the IND-CPA and KDM security of the SWHEwDD scheme. Then, we replace the PRG outputs $\mathsf{PRG}(s)$ with uniform bits. This step is computationally indistinguishable, as guaranteed by the pseudorandomness of the PRG. Finally, we replace the 5-bit communication for each AND gate with uniform bits. In the real world, these bits are computed as $d_\gamma^{i,j} = \hat{z}^{i,j} \oplus r_\gamma^{i,j}$ and $d_\gamma = \mathsf{LSB}(W_\gamma^0) \oplus \pi_\gamma = \mathsf{LSB}(W_\gamma) \oplus x_\gamma \oplus \pi_\gamma$. This replacement is statistically indistinguishable, since both $r_\gamma^{i,j}$ and $\pi_\gamma$ are uniformly distributed and independent of all other variables.

Hence, the adversary's view generated by $\mathcal{S}$ is computationally indistinguishable from the real view, which completes the proof. □

### B.3 Security Proof of BitGC-tiny

**Theorem 7** (Theorem 3, restated)**.** *Garbling scheme* BitGC-tiny *shown in Figure 8 satisfies privacy, obliviousness and authenticity, provided that the leveled HE with distributed decryption scheme satisfies the IND-CPA and KDM security properties,* PRG *is a secure PRG,* PRF *is a secure PRF, and* H *is a random oracle.*

*Sketch.* The garbling scheme BitGC-tiny achieves privacy, obliviousness, and authenticity. It combines the BitGC-small and BitGC-fast schemes: AND gates are handled as in BitGC-small, and XOR gates as in BitGC-fast. Thus, the proof follows the structure of Theorem 1, and we provide a sketch.

The simulator treats AND gates as in the proof of BitGC-small, and XOR gates as in the proof of BitGC-fast. The proof uses a standard hybrid argument. First, each input label $W_i$ is replaced with a uniform element from $\mathcal{R}_p$, which is statistically indistinguishable since the real labels are already uniform. Next, the uniform string $V_i$ is replaced with $\mathsf{H}(W_i^{1-x_i})$, which remains indistinguishable in the random-oracle model because querying $W_i^{1-x_i}$ would break the SWHEwDD scheme's IND-CPA and KDM security. Next, all ciphertexts, including $[\![\Delta]\!]$ and $[\![s_i]\!]$, are replaced with encryptions of $0$, giving computational indistinguishability by the IND-CPA and KDM security properties of the leveled HE with distributed decryption scheme. Then, the PRG outputs $\mathsf{PRG}(s)$ are replaced with uniform bits, indistinguishable by the pseudorandomness of the PRG. Finally, the one-bit communication for each AND gate is replaced with uniform bits. In the real scheme, these bits are computed as $d_\gamma = \mathsf{LSB}(W_\gamma^0) \oplus \pi_\gamma = \mathsf{LSB}(W_\gamma) \oplus x_\gamma \oplus \pi_\gamma$; this replacement is statistically indistinguishable since $\pi_\gamma$ is uniform and independent. Hence, the adversary's view under $\mathcal{S}$ is computationally indistinguishable from the real view, completing the proof. □